

GUIA TECNICA DE PROYECTOS PERSONALES

Grandes Proyectos Personales con Java + Spring Boot

30+ ideas para construir sistemas reales y aprender Backend profesional

Una guía de proyectos de dificultad creciente para practicar Java, Spring Boot, bases de datos, seguridad, arquitectura, concurrencia, sistemas distribuidos, microservicios, IA y DevOps.

Enfoque: empezar con un monolito modular bien diseñado y evolucionar solo cuando la complejidad, la carga o los límites organizacionales justifiquen nuevas fronteras.

Catalogo tecnico | 42 proyectos principales + 24 ideas secundarias

Indice

Parte 1: FinTech y sistemas financieros

1. Banco Digital
2. Billetera Digital
3. Plataforma de Pagos
4. Sistema de Prestamos
5. Personal Finance Manager
6. Plataforma de Inversiones
7. Exchange / Trading Platform Simulada
8. Sistema Antifraude

Parte 2: E-commerce y marketplaces

9. Marketplace tipo MercadoLibre
10. Tienda Online Modular
11. Plataforma de Suscripciones
12. Sistema de Delivery
13. Gestion de Inventario
14. Plataforma B2B de Compras
15. Sistema de Reservas y Cupones

Parte 3: Plataformas sociales y comunicacion

16. Red Social
17. Plataforma de Chat
18. Discord Simplificado
19. Plataforma de Comunidades
20. Sistema de Notificaciones en Tiempo Real

Parte 4: Educacion

21. Plataforma Educativa
22. Sistema de Cursos Online
23. LMS Empresarial
24. Plataforma de Exámenes
25. Sistema de Gestion Universitaria
26. Sistema de Seguimiento Academico

Parte 5: IA y programacion

27. AI Coding Platform
28. Asistente de Programacion
29. Plataforma de Ejecucion deCodigo
30. RAG sobre Documentos
31. Plataforma de Agentes IA
32. AI Code Review Platform

Parte 6: Sistemas de gran escala

33. Uber Simplificado
34. Airbnb Simplificado
35. Sistema tipo Netflix
36. Sistema tipo Spotify
37. Plataforma tipo Steam
38. Mini AWS
39. Sistema de Observabilidad
40. Plataforma de Deployment
41. Sistema Global de Pagos
42. Sistema de Event Streaming

Parte 1 - Como utilizar esta guia

Que hace que un proyecto sea "grande"

Un proyecto grande no se define por la cantidad de tablas ni por tener muchos endpoints. Se vuelve grande cuando contiene reglas de negocio, estados, permisos, integraciones, carga, procesos asincronos o restricciones que obligan a tomar decisiones de arquitectura.

```
CRUD basico
-> Aplicacion CRUD
-> Sistema empresarial
-> Plataforma
-> Sistema distribuido
```

Ejemplo FinTech

CRUD de cuentas -> banco digital -> sistema financiero -> plataforma FinTech -> arquitectura distribuida

Regla de oro

No empieces con microservicios porque "suena profesional". Primero crea limites de dominio claros dentro de un monolito modular y observa donde realmente aparecen necesidades de separacion.

Monolito modular: una sola aplicacion desplegable cuyos modulos mantienen responsabilidades y dependencias explicitas. Es una forma de aprender arquitectura sin pagar desde el inicio el costo operativo de distribuir todo.

Una ruta de crecimiento razonable

```
Nivel 1  MVP
Nivel 2  Backend completo
Nivel 3  Seguridad y calidad
Nivel 4  Cache + eventos
Nivel 5  Docker + CI/CD
Nivel 6  Microservicios selectivos
Nivel 7  Observabilidad + escalabilidad
```

Parte 2 - Catalogo de proyectos

42 proyectos principales, organizados por dominio. Cada ficha sigue la misma estructura para facilitar comparaciones técnicas.

Lectura recomendada: primero elige un dominio que te interese; despues revisa sus problemas tecnicos y su evolucion. El objetivo es convertir una idea en una secuencia de entregas concretas.

Conceptos avanzados que aparecen en el catalogo

Dependency Injection / IoC

Spring crea y conecta objetos gestionados por el contenedor. El código expresa de qué depende una clase y el framework resuelve la construcción y el ciclo de vida.

Transaction

Grupo de operaciones que debe cumplir una unidad de consistencia. En Spring suele expresarse con `@Transactional` y se apoya en las capacidades de la base de datos.

Optimistic locking

Se guarda una versión del registro y se rechaza una escritura si la versión ya cambió. Sirve para detectar conflictos sin bloquear todo el recurso.

Pessimistic locking

La base de datos mantiene un lock durante una sección de la transacción para impedir ciertas escrituras simultáneas. Puede ser útil cuando el conflicto esperado es alto.

Event-driven architecture

Un servicio publica que algo ocurrió y otros componentes reaccionan. Reduce acoplamiento temporal, pero introduce problemas de reintentos, orden y consistencia eventual.

Idempotencia

Una misma solicitud repetida produce el mismo efecto observable que procesarla una vez. Es clave cuando existen reintentos, webhooks o mensajes duplicados.

Caching

Guardar datos derivados o de lectura frecuente en una capa más rápida, como Redis. La dificultad real está en definir expiración, invalidación y consistencia.

Distributed tracing

Permite seguir una petición a través de servicios y procesos. Es especialmente útil cuando un usuario percibe lentitud pero la causa está repartida entre varios componentes.

FinTech y sistemas financieros

Los siguientes proyectos estan ordenados solo por su tematica. La guia no los clasifica como mejores o peores; cada uno permite practicar un conjunto diferente de problemas de backend.

Banco Digital

Avanzado

1. Idea general

Una plataforma bancaria educativa y realista donde un usuario puede abrir cuentas, mover dinero, administrar medios de pago y consultar un historial auditable. El valor del proyecto aparece cuando se incorporan límites, estados, idempotencia, concurrencia, conciliación y seguridad, no en el CRUD de cuentas.

2. Como funcionaria?

Cliente -> autenticación -> dashboard -> operación financiera -> validación de reglas -> transacción
SQL -> evento de negocio -> actualización de saldos / notificación.

3. Usuarios del sistema

CUSTOMER: administra cuentas y operaciones. ADMIN: configura reglas y revisa auditoría. SUPPORT: consulta operaciones y atiende incidencias.

4. Funcionalidades principales

- Apertura y cierre de cuentas
- Transferencias entre cuentas
- Límites diarios y por operación
- Historial y extractos descargables
- Bloqueo preventivo de cuenta
- Gestión de beneficiarios
- Alertas por operaciones sensibles
- Auditoría de acciones administrativas
- Conciliación de movimientos
- Idempotency keys para reintentos

5. Entidades principales

- User
- Account
- Transaction
- Transfer
- Beneficiary
- LimitRule
- Notification
- AuditLog

6. API REST

- POST /api/auth/register
- POST /api/auth/login
- GET /api/accounts
- POST /api/transfers
- GET /api/transactions
- POST /api/accounts/{id}/freeze

7. Arquitectura inicial

```
Controller -> Service -> Domain rules -> Repository -> PostgreSQL
|
+--> Event publisher -> Kafka
+--> Notification worker
```

8. Tecnologías recomendadas

Obligatorio

- Java 21
- Spring Boot
- Spring Web
- Spring Data JPA
- PostgreSQL

Recomendado

- Spring Security + JWT
- Validation
- Flyway
- Testcontainers
- Redis
- Kafka
- Docker

Avanzado

WebSockets, OpenTelemetry, API Gateway, microservices selectivos

9. Problemas tecnicos interesantes

- Concurrencia sobre saldos: dos transferencias no pueden sobrescribir cambios entre si.
- Idempotencia: un reintento HTTP no debe duplicar una transferencia.
- Transacciones: saldo, movimiento y estado deben comprometerse de forma consistente.
- Auditoria: cada cambio sensible debe quedar trazable.
- Eventual consistency: las notificaciones pueden llegar despues del commit.

10. Ejemplo de escenario real

Dos peticiones intentan debitar la misma cuenta con milisegundos de diferencia. El sistema debe impedir saldo negativo inesperado utilizando una transaccion, un mecanismo de locking y una regla de negocio centralizada.

11. Evolucion del proyecto

1. MVP: usuarios, cuentas y transferencias
2. Backend completo: JPA, validation y errores consistentes
3. Security: JWT, roles y control de acceso
4. Concurrencia: locking, idempotencia y pruebas multihilo
5. Cache + eventos: Redis y Kafka
6. Docker + CI/CD + observabilidad
7. Extraccion de servicios de pagos, notificaciones o antifraude cuando exista carga o autonomia suficiente

12. Que aprenderia

Spring Boot

* * * * *

JPA / Hibernate

* * * * *

Transactions

* * * * *

Security

* * * * *

SQL

* * * * *

Kafka

* * * * *

Concurrency

* * * * *

Observability

* * * * *

13. Dificultad

Clasificación técnica: **Avanzado**. La etiqueta expresa el tipo de problemas de ingeniería presentes; no es un ranking entre proyectos.

14. Como convertirlo en un proyecto de portfolio

README con decisiones de arquitectura, modelo ER, OpenAPI, pruebas de concurrencia, Docker Compose, CI, screenshots del dashboard y un ejemplo reproducible de una transferencia idempotente.

En proyectos reales, agregar documentación de decisiones, pruebas automatizadas y observabilidad antes de dividir el sistema en servicios.

Billetera Digital

Avanzado

1. Idea general

Una wallet donde usuarios cargan dinero, pagan, transfieren, solicitan retiros y administran fuentes de fondos. Permite practicar el diseño de un ledger y separar el saldo visible de los movimientos que explican de donde viene cada importe.

2. Como funcionaria?

Usuario -> login -> wallet -> ingreso de fondos / pago -> validacion -> ledger -> evento -> saldo materializado -> notificacion.

3. Usuarios del sistema

USER: operaciones y beneficiarios. MERCHANT: cobra. ADMIN: gestiona limites y disputas. RISK: revisa operaciones sospechosas.

4. Funcionalidades principales

- Carga de saldo
- Retiro de fondos
- Pagos a comercios
- Transferencias P2P
- Beneficiarios
- Limites y niveles de cuenta
- Historial tipo ledger
- Reversiones y refunds
- Disputas
- Notificaciones transaccionales

5. Entidades principales

- User
- Wallet
- LedgerEntry
- Payment
- Merchant
- FundingSource
- Refund
- Dispute
- Notification

6. API REST

- POST /api/wallets
- POST /api/topups
- POST /api/payments
- POST /api/transfers
- POST /api/refunds
- GET /api/ledger

7. Arquitectura inicial

Controller -> Application Service -> Ledger domain -> Repository -> PostgreSQL
+> Events -> Kafka

8. Tecnologías recomendadas

Obligatorio

- Java
- Spring Boot
- JPA
- PostgreSQL

Recomendado

- Spring Security
- Redis
- Kafka/RabbitMQ
- Flyway
- Testcontainers
- Docker

Avanzado

Outbox pattern, tracing, read models y microservicios de payments/risk

9. Problemas tecnicos interesantes

- Ledger append-only: corregir no significa borrar la operacion original.
- Idempotencia de pagos: el mismo evento no debe generar doble cargo.
- Conciliacion: el balance derivado debe coincidir con el ledger.
- Reintentos de proveedores: respuestas duplicadas deben tolerarse.
- Consistencia entre pago confirmado y notificacion.

10. Ejemplo de escenario real

Un comercio recibe timeout despues de enviar un pago. El cliente reintenta y llegan dos requests. Una clave idempotente asociada al pago permite devolver el mismo resultado sin cobrar dos veces.

11. Evolucion del proyecto

1. MVP: wallet + ledger
2. Reglas de negocio + validation
3. Security y roles
4. Idempotencia + outbox
5. Kafka + consumidores de notificaciones
6. Conciliacion y reportes
7. Separacion de payments/risk como servicios independientes

12. Que aprenderia

Transactions

* * * * *

Domain modeling

* * * * *

JPA

* * * * *

Messaging

* * * * *

Idempotency

* * * * *

Security

* * * * *

Testing

* * * * *

Distributed systems

* * * * *

13. Dificultad

Clasificación técnica: **Avanzado**. La etiqueta expresa el tipo de problemas de ingeniería presentes; no es un ranking entre proyectos.

14. Como convertirlo en un proyecto de portfolio

Mostrar un diagrama de ledger, casos de reintento, pruebas de integridad de saldo y un flujo end-to-end de pago con trazas.

En proyectos reales, agregar documentación de decisiones, pruebas automatizadas y observabilidad antes de dividir el sistema en servicios.

Plataforma de Pagos

Avanzado

1. Idea general

Un backend que recibe pagos de comercios, valida medios de pago, calcula comisiones, actualiza estados y comunica resultados a terceros. Es un excelente laboratorio para webhooks, idempotencia, estados y procesamiento asincrono.

2. Como funcionaria?

```
Merchant -> Payment API -> validation -> payment state machine -> provider adapter -> callback/
webhook -> event bus -> settlement/reporting.
```

3. Usuarios del sistema

MERCHANT: crea y consulta pagos. ADMIN: gestiona configuraciones. OPERATIONS: revisa fallos y conciliaciones.

4. Funcionalidades principales

- Creacion de payment intents
- Confirmacion de pago
- Cancelacion y refund
- Estados de pago
- Webhooks firmados
- Reintentos controlados
- Comisiones por transaccion
- Busqueda y filtros
- Conciliacion con proveedor
- API keys por comercio

5. Entidades principales

- Merchant
- Payment
- PaymentAttempt
- PaymentMethod
- WebhookEvent
- Refund
- Fee
- Settlement
- ApiKey

6. API REST

- POST /api/payments
- POST /api/payments/{id}/confirm
- POST /api/payments/{id}/refund
- GET /api/payments/{id}
- POST /api/webhooks/provider
- GET /api/settlements

7. Arquitectura inicial

```
REST API -> Payment Service -> Provider Adapter
|
+--> DB
+--> Event Bus -> Webhook / Settlement workers
```

8. Tecnologías recomendadas

Obligatorio

- Spring Boot
- Spring Web
- JPA
- PostgreSQL

Recomendado

- Security
- Redis
- Kafka
- Resilience4j
- Flyway
- OpenAPI
- Docker

Avanzado

API Gateway, service mesh, distributed tracing, microservices

9. Problemas tecnicos interesantes

- State machine: un pago no puede saltar arbitrariamente entre estados.
- Webhook duplication: el proveedor puede reintentar varias veces.
- Timeouts: un timeout no implica automaticamente fallo definitivo.
- Rate limits: proteger la API de merchants.
- Signature verification: validar que el webhook realmente proviene del proveedor.

10. Ejemplo de escenario real

El proveedor tarda 8 segundos y el cliente abandona la pantalla. El backend deja el pago en estado pendiente, procesa el callback posterior y actualiza el estado sin bloquear la peticion original.

11. Evolucion del proyecto

1. MVP: pagos internos
2. State machine + provider abstraction
3. Security + API keys
4. Webhooks + retries
5. Kafka + settlement
6. Observability + resilience
7. Separacion de payment, webhook y settlement services

12. Que aprenderia

REST APIs

* * * * *

State machines

* * * * *

Resilience

* * * * *

Webhooks

* * * * *

Kafka

* * * * *

Security

* * * * *

Distributed consistency

* * * * *

Testing

* * * * *

13. Dificultad

Clasificación técnica: **Avanzado**. La etiqueta expresa el tipo de problemas de ingeniería presentes; no es un ranking entre proyectos.

14. Como convertirlo en un proyecto de portfolio

Publicar contratos OpenAPI, ejemplos de webhook, diagrama de estados y pruebas que demuestren reintentos seguros.

En proyectos reales, agregar documentación de decisiones, pruebas automatizadas y observabilidad antes de dividir el sistema en servicios.

Sistema de Prestamos

Avanzado

1. Idea general

Plataforma donde usuarios solicitan credito, el sistema evalua reglas, calcula cuotas, crea un plan de amortizacion y gestiona pagos y mora. Permite combinar reglas de negocio, fechas, calculos monetarios y procesos programados.

2. Como funcionaria?

```
Applicant -> application -> validation -> risk rules -> offer -> acceptance -> loan account ->
installments -> payment -> delinquency workflow.
```

3. Usuarios del sistema

BORROWER: solicita y paga. ANALYST: revisa casos. ADMIN: configura productos, tasas y reglas.

4. Funcionalidades principales

- Solicitud de prestamo
- Evaluacion por reglas
- Ofertas personalizadas
- Plan de cuotas
- Calendario de vencimientos
- Registro de pagos
- Mora e intereses
- Refinanciacion
- Historial de decisiones
- Reportes de cartera

5. Entidades principales

- User
- LoanApplication
- LoanOffer
- Loan
- Installment
- Payment
- RatePlan
- RiskDecision
- DelinquencyCase

6. API REST

- POST /api/loan-applications
- GET /api/loan-applications/{id}
- POST /api/loans/{id}/accept
- GET /api/loans/{id}/schedule
- POST /api/installments/{id}/pay
- GET /api/loans?status=ACTIVE

7. Arquitectura inicial

```
Controller -> Loan Application Service -> Domain rules -> Repositories
+ -> Scheduler / events -> delinquency workers
```

8. Tecnologías recomendadas

Obligatorio

- Spring Boot
- JPA
- PostgreSQL
- Validation

Recomendado

- Spring Security
- Quartz/Scheduling
- Kafka
- Redis
- Testcontainers
- Docker

Avanzado

Rules engine, separate risk service, data platform

9. Problemas técnicos interesantes

- Precision monetaria: usar tipos adecuados y reglas explícitas para redondeo.
- Fechas: vencimientos, zonas horarias y días hábiles.
- Idempotencia de pagos.
- Procesamiento batch de cuotas vencidas.
- Auditoría de decisiones crediticias.

10. Ejemplo de escenario real

Un proceso nocturno detecta miles de cuotas vencidas mientras algunos usuarios pagan en el mismo periodo. El diseño debe evitar marcar como moroso un pago que ya fue confirmado.

11. Evolucion del proyecto

1. MVP: solicitudes + cuotas
2. Motor de reglas simple
3. Security + roles
4. Schedulers + eventos
5. Optimizar consultas y batch
6. Observability + reporting
7. Separar risk y servicing si el dominio crece

12. Que aprenderia

Business rules

* * * * *

Transactions

* * * * *

Scheduling

* * * * *

SQL

* * * * *

Security

* * * * *

Batch processing

* * * * *

Testing

* * * * *

Events

* * * * *

13. Dificultad

Clasificación técnica: **Avanzado**. La etiqueta expresa el tipo de problemas de ingeniería presentes; no es un ranking entre proyectos.

14. Como convertirlo en un proyecto de portfolio

Agregar casos de prueba de amortización, diagrama de estados de un préstamo, reglas documentadas y dataset de ejemplo.

En proyectos reales, agregar documentación de decisiones, pruebas automatizadas y observabilidad antes de dividir el sistema en servicios.

Personal Finance Manager

Intermedio/Avanzado

1. Idea general

Aplicacion para consolidar ingresos, gastos, presupuestos y metas. Aunque empieza como monolito sencillo, puede evolucionar a importacion de movimientos, categorizacion automatica, alertas y analytics.

2. Como funcionaria?

User -> import/manual entry -> categorization -> rules -> aggregates -> dashboard -> alert/event.

3. Usuarios del sistema

USER: administra sus finanzas. ADMIN: mantiene catalogos globales. El sistema puede usar integraciones de bancos en una etapa posterior.

4. Funcionalidades principales

- Registro de ingresos
- Registro de gastos
- Categorías
- Presupuestos
- Metas de ahorro
- Importacion CSV
- Reglas automaticas
- Alertas de presupuesto
- Dashboard mensual
- Exportacion de datos

5. Entidades principales

- User
- Account
- Category
- Transaction
- Budget
- BudgetPeriod
- Goal
- Rule
- ImportBatch

6. API REST

- POST /api/transactions
- GET /api/transactions
- POST /api/budgets
- GET /api/analytics/monthly
- POST /api/imports
- GET /api/goals

7. Arquitectura inicial

Flutter/Web -> Spring Boot REST -> Service -> JPA -> PostgreSQL
+> Redis for aggregates

8. Tecnologías recomendadas

Obligatorio

- Spring Boot
- REST
- JPA
- PostgreSQL

Recomendado

- Redis
- scheduler
- CSV import
- Security
- Flutter

Avanzado

Kafka, ML categorization service, warehouse/BI

9. Problemas tecnicos interesantes

- Imports duplicados: detectar archivos o movimientos repetidos.
- Agregaciones: dashboard eficiente sin recalcular todo.
- Privacidad y control de acceso por usuario.
- Consistencia de categorías durante importaciones masivas.

10. Ejemplo de escenario real

El usuario importa dos veces el mismo CSV. El sistema usa un identificador de lote y claves de deduplicacion para evitar duplicar transacciones.

11. Evolucion del proyecto

1. MVP manual
2. Importacion + filtros
3. Security
4. Redis analytics
5. Jobs + notifications
6. Flutter app
7. AI categorization as optional service

12. Que aprenderia

Spring Web

* * * * *

JPA

* * * * *

SQL

* * * * *

Caching

* * * * *

Security

* * * * *

Scheduling

* * * * *

Data processing

* * * * *

Flutter integration

* * * * *

13. Dificultad

Clasificacion tecnica: **Intermedio/Avanzado**. La etiqueta expresa el tipo de problemas de ingenieria presentes; no es un ranking entre proyectos.

14. Como convertirlo en un proyecto de portfolio

Incluir dashboard, dataset de prueba, comparativa de consultas SQL y un ejemplo de importacion idempotente.

En proyectos reales, agregar documentacion de decisiones, pruebas automatizadas y observabilidad antes de dividir el sistema en servicios.

Plataforma de Inversiones

Avanzado

1. Idea general

Sistema educativo de inversiones donde los usuarios crean carteras, compran activos simulados, reciben precios de mercado y consultan rendimiento. Puede empezar con datos falsos y luego integrar proveedores reales sin convertir el proyecto en un sistema de trading real.

2. Como funcionaria?

User -> portfolio -> order -> validation -> matching simulator -> position update -> market event -> valuation.

3. Usuarios del sistema

INVESTOR: opera carteras. ADMIN: gestiona activos y reglas. ANALYST: consulta datos y reportes.

4. Funcionalidades principales

- Carteras
- Activos
- Ordenes simuladas
- Posiciones
- Precios historicos
- Rendimiento
- Comisiones
- Watchlists
- Alertas de precio
- Reportes

5. Entidades principales

- User
- Portfolio
- Asset
- Order
- Execution
- Position
- PricePoint
- Fee
- Watchlist
- PriceAlert

6. API REST

- POST /api/portfolios
- POST /api/orders
- GET /api/orders/{id}
- GET /api/positions
- GET /api/assets/{symbol}/prices
- POST /api/alerts

7. Arquitectura inicial

Market data -> ingestion -> cache -> REST API
Order API -> trading domain -> DB -> event stream -> valuation

8. Tecnologías recomendadas

Obligatorio

- Spring Boot
- Web
- JPA
- PostgreSQL

Recomendado

- Redis
- Kafka
- WebSockets
- Security
- scheduler
- Docker

Avanzado

Time-series DB, event sourcing for selected domains, microservices

9. Problemas tecnicos interesantes

- Ordenes concurrentes y consistencia de posiciones.
- Datos de mercado de alta frecuencia en etapas avanzadas.
- Cache invalidation de precios.
- Calculo reproducible de rendimiento y comisiones.

10. Ejemplo de escenario real

Dos ordenes compran el mismo activo antes de actualizar la posicion. El servicio debe serializar o controlar el acceso al agregado afectado para no perder una ejecucion.

11. Evolucion del proyecto

1. MVP con mercado simulado
2. Portfolios + orders
3. Security + WebSockets
4. Redis + live prices
5. Kafka ingestion
6. Load testing
7. Separar market-data y order services

12. Que aprenderia

WebSockets

* * * * *

Concurrency

* * * * *

Caching

* * * * *

JPA

* * * * *

Events

* * * * *

Security

* * * * *

Performance

* * * * *

Domain modeling

* * * * *

13. Dificultad

Clasificacion tecnica: **Avanzado**. La etiqueta expresa el tipo de problemas de ingenieria presentes; no es un ranking entre proyectos.

14. Como convertirlo en un proyecto de portfolio

Agregar simulacion de mercado, graficos, decisiones de arquitectura, pruebas de concurrencia y un panel de monitoreo de eventos.

En proyectos reales, agregar documentacion de decisiones, pruebas automatizadas y observabilidad antes de dividir el sistema en servicios.

Exchange / Trading Platform Simulada

Muy Avanzado

1. Idea general

Un exchange de laboratorio con order book, ordenes limit/market, matching engine simplificado y balances virtuales. Es ideal para aprender concurrencia, estructuras de datos, eventos y limites de latencia sin manejar dinero real.

2. Como funcionaria?

```
Client -> order API -> validation -> matching engine -> execution -> ledger -> market event ->
WebSocket clients.
```

3. Usuarios del sistema

TRADER: envia ordenes. ADMIN: configura mercados. MONITOR: observa libro y metricas.

4. Funcionalidades principales

- Order book
- Limit orders
- Market orders
- Cancelacion
- Matching
- Historial de trades
- Balances virtuales
- Comisiones
- Market snapshots
- WebSocket live feed

5. Entidades principales

- User
- Market
- Order
- OrderBook
- Trade
- Balance
- Fee
- MarketSnapshot

6. API REST

- POST /api/markets/{symbol}/orders
- DELETE /api/orders/{id}
- GET /api/order-books/{symbol}
- GET /api/trades
- GET /api/balances
- WS /ws/markets/{symbol}

7. Arquitectura inicial

```
REST -> Order Gateway -> Matching Engine -> Execution Store
      +-> Event Stream -> WebSocket
      +-> Ledger
```

8. Tecnologías recomendadas

Obligatorio

- Java
- Spring Boot
- PostgreSQL

Recomendado

- WebSockets
- Redis
- Kafka
- JMH
- Testcontainers
- Docker

Avanzado

Disruptor-like patterns, dedicated matching process, horizontal market partitioning

9. Problemas tecnicos interesantes

- Race conditions en el order book.
- Orden parcial y prioridad temporal.
- Backpressure si llegan muchas actualizaciones.
- Consistencia entre trade y balances.
- Persistencia sin bloquear el camino critico.

10. Ejemplo de escenario real

Dos ordenes cruzan el mismo precio al mismo tiempo. El motor debe respetar una regla determinista de prioridad y producir una secuencia de ejecuciones consistente.

11. Evolucion del proyecto

1. MVP in-memory
2. Persistencia de ordenes
3. Concurrencia controlada
4. WebSockets + Redis
5. Kafka event stream
6. Benchmarks + profiling
7. Particionado por mercado

12. Que aprenderia

Concurrency

* * * * *

Data structures

* * * * *

WebSockets

* * * * *

Kafka

* * * * *

Performance

* * * * *

Transactions

* * * * *

Testing

* * * * *

Distributed design

* * * * *

13. Dificultad

Clasificación técnica: **Muy Avanzado**. La etiqueta expresa el tipo de problemas de ingeniería presentes; no es un ranking entre proyectos.

14. Como convertirlo en un proyecto de portfolio

Documentar invariantes del matching engine, benchmarks, pruebas de concurrencia y una UI que muestre el order book en tiempo real.

En proyectos reales, agregar documentación de decisiones, pruebas automatizadas y observabilidad antes de dividir el sistema en servicios.

Sistema Antifraude

Avanzado

1. Idea general

Servicio que evalua transacciones usando reglas, listas, limites y modelos opcionales para detectar comportamiento sospechoso. Es un proyecto de backend muy bueno para combinar sincronismo y asincronismo.

2. Como funcionaria?

Transaction -> risk API -> rules -> features -> decision -> approve/review/reject -> event -> case management.

3. Usuarios del sistema

SYSTEM: envia transacciones. ANALYST: revisa alertas. ADMIN: configura reglas y listas.

4. Funcionalidades principales

- Scoring de riesgo
- Reglas por monto
- Velocity checks
- Listas de bloqueo
- Alertas
- Casos de investigacion
- Notas de analista
- Feedback de resultados
- Explicacion de decision
- Metricas de fraude

5. Entidades principales

- Transaction
- RiskDecision
- RiskRule
- WatchlistEntry
- FraudCase
- AnalystNote
- RiskFeature
- ModelVersion

6. API REST

- POST /api/risk/evaluate
- GET /api/risk/decisions/{id}
- GET /api/fraud-cases
- POST /api/fraud-cases/{id}/resolve
- POST /api/rules
- GET /api/watchlists

7. Arquitectura inicial

Transaction API -> Risk Engine -> Rules + Feature Store -> Decision
+> Event Bus -> Case Service

8. Tecnologías recomendadas

Obligatorio

- Spring Boot
- PostgreSQL
- Redis

Recomendado

- Kafka
- Spring Security
- scheduled jobs
- Docker
- Python model service

Avanzado

Streaming features, model registry, feature store, microservice extraction

9. Problemas tecnicos interesantes

- Baja latencia en la decision.
- Features recientes y cache.
- Falsos positivos y explicabilidad.
- Versionado de reglas/modelos.
- Eventos fuera de orden.

10. Ejemplo de escenario real

Una cuenta realiza 20 operaciones pequeñas en pocos segundos. Un velocity rule detecta el patron y deriva la siguiente operacion a revision sin bloquear la logica de auditoria.

11. Evolucion del proyecto

1. Rules MVP
2. Risk scores + Redis
3. Cases + audit
4. Kafka streaming
5. Model service
6. Observability
7. Scalable risk partitions

12. Que aprenderia

Redis

* * * * *

Kafka

* * * *

Rules

* * * *

Security

* * * * *

Events

* * * *

Performance

* * * *

AI integration

* * * * *

Observability

* * * *

13. Dificultad

Clasificacion tecnica: **Avanzado**. La etiqueta expresa el tipo de problemas de ingenieria presentes; no es un ranking entre proyectos.

14. Como convertirlo en un proyecto de portfolio

Mostrar reglas versionadas, ejemplos de decisiones explicables, dashboard de casos y metricas de latencia.

En proyectos reales, agregar documentacion de decisiones, pruebas automatizadas y observabilidad antes de dividir el sistema en servicios.

E-commerce y marketplaces

Los siguientes proyectos estan ordenados solo por su tematica. La guia no los clasifica como mejores o peores; cada uno permite practicar un conjunto diferente de problemas de backend.

Marketplace tipo MercadoLibre

Avanzado

1. Idea general

Marketplace multi-vendedor con catalogo, publicaciones, checkout, pagos simulados, envios y reputacion. La complejidad aparece porque una compra cruza varios dominios y actores.

2. Como funcionaria?

```
Buyer -> search -> product -> cart -> checkout -> payment -> inventory reservation -> shipment -> notifications -> rating.
```

3. Usuarios del sistema

BUYER: compra. SELLER: publica y despacha. ADMIN: modera. SUPPORT: gestiona disputas.

4. Funcionalidades principales

- Catalogo
- Busqueda y filtros
- Carrito
- Checkout
- Pagos
- Reserva de stock
- Seguimiento de envio
- Reputacion
- Cupones
- Disputas

5. Entidades principales

- User
- SellerProfile
- Product
- Listing
- Cart
- Order
- OrderItem
- Inventory
- Shipment
- Review
- Coupon
- Dispute

6. API REST

- GET /api/products
- POST /api/carts/items
- POST /api/orders
- POST /api/orders/{id}/pay
- GET /api/shipments/{id}
- POST /api/reviews

7. Arquitectura inicial

```
API -> Catalog/Order modules -> PostgreSQL
      Event Bus -> Inventory / Payment / Shipment / Notification
```

8. Tecnologías recomendadas

Obligatorio

- Spring Boot
- JPA
- PostgreSQL
- Security

Recomendado

- Redis
- Kafka
- Elasticsearch/OpenSearch
- Docker
- Flutter

Avanzado

API Gateway + microservices for search/order/shipping

9. Problemas tecnicos interesantes

- Stock overselling por concurrencia.
- Checkout parcialmente fallido.
- Búsqueda escalable.
- Eventos duplicados y reintentos.

10. Ejemplo de escenario real

Dos usuarios compran la ultima unidad. La reserva de stock debe ser atomica o estar protegida por versionado/locking.

11. Evolucion del proyecto

1. MVP catalogo
2. Cart + order
3. Security + payment sandbox
4. Inventory reservation
5. Search + Redis
6. Kafka + workers
7. Service extraction

12. Que aprenderia

JPA

* * * * *

Security

* * * *

Transactions

* * * *

Caching

* * * * *

Messaging

* * * *

Search

* * * *

Concurrency

* * * * *

Architecture

* * * *

13. Dificultad

Clasificación técnica: **Avanzado**. La etiqueta expresa el tipo de problemas de ingeniería presentes; no es un ranking entre proyectos.

14. Como convertirlo en un proyecto de portfolio

Demostrar la carrera de stock con un test reproducible, OpenAPI, Docker Compose y un frontend Flutter.

En proyectos reales, agregar documentacion de decisiones, pruebas automatizadas y observabilidad antes de dividir el sistema en servicios.

Tienda Online Modular

Intermedio/Avanzado

1. Idea general

E-commerce de una sola empresa con catalogo, checkout y postventa. Es un punto de entrada ideal para construir un monolito modular serio antes de llegar a un marketplace.

2. Como funcionaria?

Customer -> catalog -> cart -> checkout -> payment -> order -> fulfillment -> refund.

3. Usuarios del sistema

CUSTOMER, ADMIN, WAREHOUSE, SUPPORT.

4. Funcionalidades principales

- Catalogo
- Variantes de producto
- Carrito
- Checkout
- Pedidos
- Descuentos
- Inventario
- Refunds
- Emails
- Reportes de ventas

5. Entidades principales

- Product
- Variant
- Cart
- Order
- OrderItem
- Inventory
- Discount
- Refund
- Customer

6. API REST

- GET /api/products
- POST /api/cart/items
- POST /api/checkout
- GET /api/orders
- POST /api/orders/{id}/refund
- GET /api/reports/sales

7. Arquitectura inicial

Controller -> Application services -> Modules (catalog/order/inventory) -> JPA

8. Tecnologías recomendadas

Obligatorio

- Spring Boot
- JPA
- PostgreSQL

Recomendado

- Security
- Redis
- Kafka
- Docker
- OpenAPI
- Flutter

Avanzado

Hexagonal architecture + modular deployment

9. Problemas tecnicos interesantes

- Checkout transactional.
- Stock consistente.
- Modularidad sin dependencias circulares.
- Cache de catalogo.

10. Ejemplo de escenario real

Un descuento expira durante el checkout. La regla debe evaluarse en el momento correcto y quedar registrada en la orden.

11. Evolucion del proyecto

1. Catalogo
2. Cart + orders
3. Security
4. Modular monolith
5. Redis + events
6. Testing + CI
7. Optional service extraction

12. Que aprenderia

Modular design

* * * * *

JPA

* * * * *

REST

* * * * *

Security

* * * * *

Transactions

* * * * *

Caching

* * * * *

Testing

* * * * *

CI/CD

* * * * *

13. Dificultad

Clasificacion tecnica: **Intermedio/Avanzado**. La etiqueta expresa el tipo de problemas de ingenieria presentes; no es un ranking entre proyectos.

14. Como convertirlo en un proyecto de portfolio

El foco debe estar en la estructura modular, pruebas y documentacion de decisiones, no en la cantidad de endpoints.

En proyectos reales, agregar documentacion de decisiones, pruebas automatizadas y observabilidad antes de dividir el sistema en servicios.

Plataforma de Suscripciones

Avanzado

1. Idea general

Gestion de productos recurrentes, ciclos de facturación, upgrades, downgrades, trials y cancelaciones. Es un gran caso para jobs, pagos, estados y consistencia.

2. Como funcionaria?

```
User -> plan -> subscribe -> billing schedule -> charge -> invoice -> retry/dunning -> renewal/cancel.
```

3. Usuarios del sistema

CUSTOMER, ADMIN, BILLING OPERATOR.

4. Funcionalidades principales

- Planes
- Trials
- Suscripciones
- Prorrato
- Facturas
- Renovaciones
- Reintentos de pago
- Cancelaciones
- Cupones
- Historial de billing

5. Entidades principales

- Customer
- Plan
- Subscription
- Invoice
- InvoiceItem
- PaymentAttempt
- Coupon
- BillingCycle

6. API REST

- POST /api/subscriptions
- PATCH /api/subscriptions/{id}
- POST /api/subscriptions/{id}/cancel
- GET /api/invoices
- POST /api/payments/{id}/retry
- GET /api/plans

7. Arquitectura inicial

```
REST -> Subscription domain -> Scheduler -> Billing -> Payment provider adapter -> Events
```

8. Tecnologías recomendadas

Obligatorio

- Spring Boot
- JPA
- PostgreSQL

Recomendado

- Scheduler
- Security
- Kafka
- Redis
- Docker

Avanzado

Temporal/workflow engine, microservices

9. Problemas técnicos interesantes

- Reintentos de cobro.
- Prorrato y dinero.
- Jobs ejecutados dos veces.
- Idempotencia de renovaciones.

10. Ejemplo de escenario real

El proceso de renovación corre dos veces por una caída del worker. Un lock distribuido o una clave única sobre el ciclo de facturación evita generar dos facturas.

11. Evolución del proyecto

1. MVP plans
2. Subscription lifecycle
3. Billing jobs
4. Payment adapter
5. Events + notifications
6. Redis + metrics
7. Separate billing service

12. Qué aprendería

Scheduling

* * * * *

Transactions

* * * * *

State machines

* * * * *

Idempotency

* * * * *

Events

* * * * *

Security

* * * * *

Money handling

* * * * *

Testing

* * * * *

13. Dificultad

Clasificación técnica: **Avanzado**. La etiqueta expresa el tipo de problemas de ingeniería presentes; no es un ranking entre proyectos.

14. Como convertirlo en un proyecto de portfolio

Incluir timeline de una suscripcion, casos de prorrato, reintentos y prueba de ejecucion duplicada.

En proyectos reales, agregar documentacion de decisiones, pruebas automatizadas y observabilidad antes de dividir el sistema en servicios.

Sistema de Delivery

Avanzado

1. Idea general

Plataforma que coordina pedidos, repartidores, asignacion y tracking. Combina estados, geolocalizacion, tiempo real y eventos.

2. Como funcionaria?

```
Customer -> order -> merchant accepts -> dispatch -> courier matching -> tracking -> delivered -> rating.
```

3. Usuarios del sistema

CUSTOMER, MERCHANT, COURIER, OPS ADMIN.

4. Funcionalidades principales

- Pedidos
- Asignacion de repartidor
- Tracking GPS
- Estados
- ETA
- Historial
- Promociones
- Problemas de entrega
- Rating
- Panel de operaciones

5. Entidades principales

- Order
- Merchant
- Courier
- Delivery
- LocationPing
- Assignment
- Customer
- Rating
- Incident

6. API REST

- POST /api/orders
- POST /api/deliveries/{id}/accept
- POST /api/couriers/{id}/location
- GET /api/orders/{id}/tracking
- WS /ws/deliveries/{id}
- POST /api/incidents

7. Arquitectura inicial

```
Mobile -> API -> Order/Dispatch modules -> DB  
+ -> Redis Geo + WebSocket + Events
```

8. Tecnologías recomendadas

Obligatorio

- Spring Boot
- PostgreSQL
- Redis

Recomendado

- WebSockets
- Kafka
- geospatial queries
- Flutter
- Docker

Avanzado

Location streaming, separate dispatch service, tracing

9. Problemas tecnicos interesantes

- Asignacion concurrente de courier.
- Ubicaciones frecuentes.
- Backpressure de eventos.
- Consistencia del estado del pedido.

10. Ejemplo de escenario real

Dos pedidos intentan reservar al mismo repartidor. El servicio de dispatch debe realizar la asignacion con una operacion atomica o lock apropiado.

11. Evolucion del proyecto

1. Orders MVP
2. Courier app
3. Tracking
4. Redis Geo + WebSockets
5. Kafka dispatch events
6. Load testing
7. Dispatch service extraction

12. Que aprenderia

Redis

* * * * *

WebSockets

* * * * *

Events

* * * * *

Concurrency

* * * * *

Geospatial data

* * * * *

REST

* * * * *

Flutter integration

* * * * *

Observability

* * * * *

13. Dificultad

Clasificacion tecnica: **Avanzado**. La etiqueta expresa el tipo de problemas de ingenieria presentes; no es un ranking entre proyectos.

14. Como convertirlo en un proyecto de portfolio

Mostrar mapa/tracking, arquitectura de dispatch, eventos y una prueba de asignacion concurrente.

En proyectos reales, agregar documentacion de decisiones, pruebas automatizadas y observabilidad antes de dividir el sistema en servicios.

Gestion de Inventario

Avanzado

1. Idea general

Sistema para controlar stock, movimientos, almacenes, reservas, reposicion y auditoria. Puede ser backend de muchos otros proyectos y es excelente para practicar consistencia de datos.

2. Como funcionaria?

```
Order/import -> inventory command -> stock validation -> reservation/movement -> event -> reorder/
reporting.
```

3. Usuarios del sistema

WAREHOUSE, ADMIN, PURCHASING, AUDITOR.

4. Funcionalidades principales

- Productos y SKUs
- Multi-almacen
- Movimientos
- Reservas
- Ajustes
- Transferencias entre almacenes
- Stock minimo
- Reposicion
- Auditoria
- Reportes

5. Entidades principales

- Product
- Sku
- Warehouse
- StockItem
- StockMovement
- Reservation
- PurchaseOrder
- Supplier
- AuditLog

6. API REST

- GET /api/stock
- POST /api/reservations
- POST /api/movements
- POST /api/transfers
- GET /api/warehouses/{id}/stock
- GET /api/reorders

7. Arquitectura inicial

```
Inventory Controller -> Service -> domain invariants -> Repository -> PostgreSQL
+--> events
```

8. Tecnologías recomendadas

Obligatorio

- Spring Boot
- JPA
- PostgreSQL

Recomendado

- Kafka
- Redis
- locking
- Batch
- Docker

Avanzado

Event-sourced subset, inventory service extraction

9. Problemas tecnicos interesantes

- Stock negativo.
- Reservas que expiran.
- Actualizaciones concurrentes.
- Reconciliacion de inventario fisico vs sistema.

10. Ejemplo de escenario real

Un pedido reserva stock y el operador ajusta manualmente la misma SKU. El dominio debe definir como se serializan o validan ambas operaciones.

11. Evolucion del proyecto

1. Stock CRUD
2. Reservations
3. Multi-warehouse
4. Audit + events
5. Batch reordering
6. Performance
7. Service extraction

12. Que aprenderia

Transactions

* * * * *

Locking

* * * * *

JPA

* * * * *

Batch

* * * * *

Events

* * * * *

SQL

* * * * *

Domain modeling

* * * * *

Testing

* * * * *

13. Dificultad

Clasificacion tecnica: **Avanzado**. La etiqueta expresa el tipo de problemas de ingenieria presentes; no es un ranking entre proyectos.

14. Como convertirlo en un proyecto de portfolio

Un buen portfolio muestra invariantes de stock, pruebas de concurrencia y reportes auditables.

En proyectos reales, agregar documentacion de decisiones, pruebas automatizadas y observabilidad antes de dividir el sistema en servicios.

Plataforma B2B de Compras

Avanzado

1. Idea general

Portal de compras entre empresas con catalogos privados, listas de precios, aprobaciones y facturacion. La complejidad esta en permisos y workflows organizacionales.

2. Como funcionaria?

Employee -> quote/cart -> approval policy -> order -> invoice -> fulfillment -> account statement.

3. Usuarios del sistema

BUYER, APPROVER, SUPPLIER, FINANCE ADMIN.

4. Funcionalidades principales

- Organizaciones
- Roles por empresa
- Catalogos privados
- Precios por cliente
- Ordenes de compra
- Aprobaciones
- Facturacion
- Limites de gasto
- Historial
- Reportes

5. Entidades principales

- Organization
- User
- Role
- Product
- PriceList
- PurchaseOrder
- Approval
- Invoice
- SpendingLimit

6. API REST

- POST /api/organizations
- GET /api/catalog
- POST /api/purchase-orders
- POST /api/purchase-orders/{id}/approve
- GET /api/invoices
- GET /api/reports/spend

7. Arquitectura inicial

API -> IAM / Catalog / Ordering modules -> PostgreSQL
+> Approval events

8. Tecnologías recomendadas

Obligatorio

- Spring Boot
- Security
- JPA
- PostgreSQL

Recomendado

- Redis
- Kafka
- audit
- Docker

Avanzado

Policy engine, service extraction, SSO/OAuth2

9. Problemas técnicos interesantes

- Autorización por tenant.
- Workflows de aprobación.
- Aislamiento de datos entre empresas.
- Cambios de precio durante checkout.

10. Ejemplo de escenario real

Un empleado intenta aprobar su propia compra cuando la política exige dos niveles. El backend debe aplicar la política de forma centralizada.

11. Evolución del proyecto

1. Single tenant
2. Multi-tenant logical
3. Approval workflow
4. Audit + events
5. Redis
6. SSO/OAuth2
7. Separate services

12. Qué aprendería

Spring Security

* * * * *

Multi-tenancy

* * * * *

Workflow

* * * * *

JPA

* * * * *

Events

* * * * *

Authorization

* * * * *

Testing

* * * * *

Architecture

* * * * *

13. Dificultad

Clasificación técnica: **Avanzado**. La etiqueta expresa el tipo de problemas de ingeniería presentes; no es un ranking entre proyectos.

14. Cómo convertirlo en un proyecto de portfolio

Documentar tenant isolation, matriz de permisos y un workflow de aprobación reproducible.

En proyectos reales, agregar documentación de decisiones, pruebas automatizadas y observabilidad antes de dividir el sistema en servicios.

Sistema de Reservas y Cupones

Avanzado

1. Idea general

Plataforma de reservas para servicios con cupos, disponibilidad, pagos y cupones. Es especialmente útil para practicar locking, expiración y reglas temporales.

2. Como funcionaria?

```
User -> search availability -> hold slot -> checkout -> confirm -> reminder -> completed/cancelled.
```

3. Usuarios del sistema

CUSTOMER, PROVIDER, ADMIN.

4. Funcionalidades principales

- Disponibilidad
- Reservas
- Holds temporales
- Pagos
- Cupones
- Cancelaciones
- Recordatorios
- Calendario
- Lista de espera
- Historial

5. Entidades principales

- User
- Provider
- Service
- Availability
- Reservation
- Hold
- Coupon
- Payment
- Reminder
- WaitlistEntry

6. API REST

- GET /api/availability
- POST /api/holds
- POST /api/reservations
- POST /api/reservations/{id}/cancel
- POST /api/coupons/validate
- GET /api/calendar

7. Arquitectura inicial

```
REST -> Availability/Booking service -> DB  
Scheduler -> hold expiration / reminders
```

8. Tecnologías recomendadas

Obligatorio

- Spring Boot
- JPA
- PostgreSQL

Recomendado

- Redis
- Scheduler
- Security
- WebSockets
- Docker

Avanzado

Distributed locks, Kafka, dedicated availability service

9. Problemas tecnicos interesantes

- Overbooking.
- Expiracion de holds.
- Timezone.
- Cupones aplicados bajo reglas temporales.

10. Ejemplo de escenario real

Dos usuarios seleccionan el mismo turno casi al mismo tiempo. El sistema crea un hold exclusivo con expiracion y solo uno puede confirmar.

11. Evolucion del proyecto

1. Availability
2. Booking
3. Holds
4. Payment + coupons
5. Reminders + Redis
6. WebSockets
7. Scalable booking service

12. Que aprenderia

Concurrency

* * * * *

Scheduling

* * * * *

Caching

* * * * *

Transactions

* * * * *

REST

* * * * *

Security

* * * * *

Time handling

* * * * *

Testing

* * * * *

13. Dificultad

Clasificacion tecnica: **Avanzado**. La etiqueta expresa el tipo de problemas de ingenieria presentes; no es un ranking entre proyectos.

14. Como convertirlo en un proyecto de portfolio

Mostrar como se evita overbooking, junto con tests de concurrencia y un calendario interactivo.

En proyectos reales, agregar documentacion de decisiones, pruebas automatizadas y observabilidad antes de dividir el sistema en servicios.

Plataformas sociales y comunicacion

Los siguientes proyectos estan ordenados solo por su tematica. La guia no los clasifica como mejores o peores; cada uno permite practicar un conjunto diferente de problemas de backend.

Red Social

Avanzado

1. Idea general

Red social con perfiles, follows, publicaciones, comentarios, reacciones y feed. El reto no es guardar posts sino construir un feed eficiente y controlar visibilidad, moderacion y notificaciones.

2. Como funcionaria?

User -> post -> moderation -> feed fan-out -> interactions -> notifications -> analytics.

3. Usuarios del sistema

USER, MODERATOR, ADMIN.

4. Funcionalidades principales

- Perfiles
- Follow/unfollow
- Posts
- Media metadata
- Likes
- Comments
- Feed
- Privacy settings
- Reports
- Notifications

5. Entidades principales

- User
- Profile
- Follow
- Post
- Comment
- Reaction
- FeedItem
- Notification
- Report

6. API REST

- POST /api/posts
- GET /api/feed
- POST /api/users/{id}/follow
- POST /api/posts/{id}/comments
- POST /api/posts/{id}/reactions
- GET /api/notifications

7. Arquitectura inicial

Spring Boot API -> Social modules -> PostgreSQL
 +-> Redis feed cache
 +-> Kafka notifications

8. Tecnologías recomendadas

Obligatorio

- Spring Boot
- JPA
- PostgreSQL

Recomendado

- Redis
- Kafka
- object storage
- Security
- WebSockets

Avanzado

Search cluster, feed service, media pipeline

9. Problemas tecnicos interesantes

- Feed fan-out and pagination.
- Privacy checks por post.
- Hot users generating many events.
- Moderation async.

10. Ejemplo de escenario real

Una cuenta con miles de seguidores publica contenido. Generar el feed para cada seguidor sin bloquear el request requiere procesamiento asincrono y cache.

11. Evolucion del proyecto

1. Profiles + posts
2. Interactions
3. Feed query
4. Redis
5. Kafka notifications
6. Moderation/search
7. Feed microservice

12. Que aprenderia

JPA

* * * * *

Caching

* * * * *

Kafka

* * * * *

Pagination

* * * * *

Security

* * * * *

Async processing

* * * * *

Search

* * * * *

Scalability

* * * * *

13. Dificultad

Clasificacion tecnica: **Avanzado**. La etiqueta expresa el tipo de problemas de ingenieria presentes; no es un ranking entre proyectos.

14. Como convertirlo en un proyecto de portfolio

El portfolio debe explicar la estrategia de feed y mostrar pruebas de paginacion y rendimiento.

En proyectos reales, agregar documentacion de decisiones, pruebas automatizadas y observabilidad antes de dividir el sistema en servicios.

Plataforma de Chat

Avanzado

1. Idea general

Chat 1 a 1 y grupal con mensajes persistentes, presencia y lectura. Es un proyecto directo para estudiar WebSockets, eventos y escalabilidad de conexiones.

2. Como funcionaria?

```
Client -> WebSocket connection -> chat service -> message store -> event -> recipients -> delivery/
read receipt.
```

3. Usuarios del sistema

USER, ADMIN/MODERATOR.

4. Funcionalidades principales

- Chats privados
- Grupos
- Mensajes
- Adjuntos
- Typing indicators
- Read receipts
- Presence
- Mute/archive
- Search
- Reportes

5. Entidades principales

- User
- Conversation
- Membership
- Message
- Attachment
- ReadReceipt
- Presence
- Report

6. API REST

- POST /api/conversations
- GET /api/conversations
- GET /api/conversations/{id}/messages
- POST /api/messages
- WS /ws/chat
- POST /api/messages/{id}/read

7. Arquitectura inicial

```
Flutter -> WebSocket Gateway -> Chat service -> PostgreSQL
+ -> Redis presence
+ -> Kafka events
```

8. Tecnologías recomendadas

Obligatorio

- Spring Boot WebSocket
- JPA
- PostgreSQL

Recomendado

- Redis
- Kafka
- Security
- object storage
- Flutter

Avanzado

Dedicated websocket gateway, pub/sub, horizontal scaling

9. Problemas tecnicos interesantes

- Conexiones persistentes.
- Orden de mensajes.
- Entrega duplicada.
- Escalado de multiples instancias.

10. Ejemplo de escenario real

El usuario pierde internet y reconecta. El servidor debe recuperar mensajes no leídos y evitar duplicar los que ya fueron confirmados.

11. Evolucion del proyecto

1. REST chat
2. WebSockets
3. Presence
4. Redis
5. Kafka
6. Horizontal scaling
7. Dedicated gateway

12. Que aprenderia

WebSockets

* * * * *

Redis

* * * * *

Kafka

* * * * *

Concurrency

* * * * *

Security

* * * * *

Real-time systems

* * * * *

Persistence

* * * * *

Flutter integration

* * * * *

13. Dificultad

Clasificación técnica: **Avanzado**. La etiqueta expresa el tipo de problemas de ingeniería presentes; no es un ranking entre proyectos.

14. Como convertirlo en un proyecto de portfolio

Demostrar chat multiusuario, reconexion, read receipts y diagrama de infraestructura.

En proyectos reales, agregar documentacion de decisiones, pruebas automatizadas y observabilidad antes de dividir el sistema en servicios.

Discord Simplificado

Avanzado

1. Idea general

Servidor de comunidades con servidores, canales, roles, permisos y chat en tiempo real. Es ideal para aprender autorizacion jerarquica y eventos en tiempo real.

2. Como funcionaria?

User -> server -> role/channel permission -> websocket -> message -> event -> notifications.

3. Usuarios del sistema

MEMBER, MODERATOR, SERVER_ADMIN, PLATFORM_ADMIN.

4. Funcionalidades principales

- Servers
- Channels
- Roles
- Permisos
- Mensajes
- Mentions
- Presence
- Invites
- Moderation
- Audit log

5. Entidades principales

- User
- Server
- Membership
- Role
- Permission
- Channel
- Message
- Invite
- AuditLog

6. API REST

- POST /api/servers
- POST /api/servers/{id}/channels
- POST /api/servers/{id}/roles
- POST /api/channels/{id}/messages
- WS /ws/servers/{id}
- GET /api/servers/{id}/audit

7. Arquitectura inicial

API + WebSocket -> Permission service/module -> PostgreSQL -> Redis/Kafka

8. Tecnologías recomendadas

Obligatorio

- Spring Boot
- PostgreSQL
- Security

Recomendado

- WebSockets
- Redis
- Kafka
- Flutter/Web
- Docker

Avanzado

Distributed gateway, permission cache, service extraction

9. Problemas tecnicos interesantes

- Permisos combinados de server y channel.
- Presence a escala.
- Orden de eventos.
- Moderacion y auditoria.

10. Ejemplo de escenario real

Un usuario es moderador global pero no tiene permiso en un canal privado. La autorizacion debe evaluar la jerarquia correcta antes de aceptar el mensaje.

11. Evolucion del proyecto

1. Servers + roles
2. Channels + permissions
3. Chat
4. WebSockets
5. Redis presence
6. Kafka events
7. Gateway scaling

12. Que aprenderia

Security

* * * * *

WebSockets

* * * * *

Redis

* * * * *

Kafka

* * * * *

RBAC

* * * * *

Authorization

* * * * *

Distributed systems

* * * * *

Testing

* * * * *

13. Dificultad

Clasificacion tecnica: **Avanzado**. La etiqueta expresa el tipo de problemas de ingenieria presentes; no es un ranking entre proyectos.

14. Como convertirlo en un proyecto de portfolio

Incluir matriz de permisos y pruebas de autorizacion, no solo pantallas de chat.

En proyectos reales, agregar documentacion de decisiones, pruebas automatizadas y observabilidad antes de dividir el sistema en servicios.

Plataforma de Comunidades

Avanzado

1. Idea general

Sistema para crear comunidades tematicas con posts, moderacion, reglas y reputacion. Combina foro, roles y ranking de contenido.

2. Como funcionaria?

User -> join community -> post/comment -> moderation -> ranking -> notification.

3. Usuarios del sistema

MEMBER, MODERATOR, COMMUNITY_ADMIN, PLATFORM_ADMIN.

4. Funcionalidades principales

- Comunidades
- Posts
- Comentarios
- Votaciones
- Moderacion
- Reglas
- Reputacion
- Tags
- Busqueda
- Notificaciones

5. Entidades principales

- Community
- Membership
- Rule
- Post
- Comment
- Vote
- ReputationEvent
- ModerationCase
- Tag

6. API REST

- POST /api/communities
- POST /api/communities/{id}/posts
- POST /api/posts/{id}/votes
- POST /api/moderation/cases
- GET /api/search
- GET /api/communities/{id}/feed

7. Arquitectura inicial

Modular monolith -> events for reputation, notifications and ranking

8. Tecnologías recomendadas

Obligatorio

- Spring Boot
- JPA
- PostgreSQL

Recomendado

- Redis
- Kafka
- search engine
- Security
- Docker

Avanzado

Ranking service, moderation service

9. Problemas tecnicos interesantes

- Ranking eficiente.
- Moderacion consistente.
- Spam/rate limits.
- Reputation derived from events.

10. Ejemplo de escenario real

Un post recibe miles de votos durante un pico. El ranking no deberia recalcular toda la comunidad por cada click; puede acumular eventos y actualizar una vista materializada.

11. Evolucion del proyecto

1. Posts
2. Moderation
3. Ranking
4. Redis
5. Kafka
6. Search
7. Separate ranking service

12. Que aprenderia

Events

* * * * *

Caching

* * * * *

Search

* * * * *

Security

* * * * *

Data modeling

* * * * *

Async

* * * * *

Performance

* * * * *

Testing

* * * * *

13. Dificultad

Clasificacion tecnica: **Avanzado**. La etiqueta expresa el tipo de problemas de ingenieria presentes; no es un ranking entre proyectos.

14. Como convertirlo en un proyecto de portfolio

Documentar ranking, moderacion y un ejemplo de procesamiento por eventos.

En proyectos reales, agregar documentacion de decisiones, pruebas automatizadas y observabilidad antes de dividir el sistema en servicios.

Sistema de Notificaciones en Tiempo Real

Avanzado

1. Idea general

Servicio reutilizable que recibe eventos y los entrega por WebSocket, email, push o in-app con preferencias, prioridad y reintentos.

2. Como funcionaria?

Business service -> notification event -> router -> channel adapter -> delivery -> retry/dead-letter -> analytics.

3. Usuarios del sistema

SYSTEM producers, USER recipients, ADMIN operators.

4. Funcionalidades principales

- Plantillas
- Canales
- Preferencias
- Prioridades
- Deduplicacion
- Reintentos
- DLQ
- Programacion
- Delivery status
- Metricas

5. Entidades principales

- Notification
- Template
- Preference
- DeliveryAttempt
- ChannelConfig
- ScheduledNotification
- DeadLetterItem

6. API REST

- POST /api/notifications
- GET /api/notifications
- POST /api/templates
- PATCH /api/preferences
- GET /api/deliveries/{id}
- WS /ws/notifications

7. Arquitectura inicial

Producer -> Kafka -> Notification router -> Channel workers -> Delivery stores

8. Tecnologías recomendadas

Obligatorio

- Spring Boot
- Kafka
- PostgreSQL

Recomendado

- Redis
- WebSockets
- email/push adapters
- Docker

Avanzado

Schema registry, DLQ platform, dedicated messaging services

9. Problemas técnicos interesantes

- Backpressure.
- Deduplicación.
- Retry policies.
- Poison messages.
- Idempotent delivery status.

10. Ejemplo de escenario real

Un consumer falla diez veces procesando un evento. El sistema lo envía a una dead-letter queue para evitar bloquear el resto.

11. Evolución del proyecto

1. In-app notifications
2. Async queue
3. Email adapter
4. WebSockets
5. Kafka + DLQ
6. Templates + preferences
7. Standalone notification platform

12. Qué aprendería

Kafka

* * * * *

Async processing

* * * * *

WebSockets

* * * * *

Redis

* * * * *

Resilience

* * * * *

Observability

* * * * *

Distributed systems

* * * * *

Testing

* * * * *

13. Dificultad

Clasificación técnica: **Avanzado**. La etiqueta expresa el tipo de problemas de ingeniería presentes; no es un ranking entre proyectos.

14. Como convertirlo en un proyecto de portfolio

Mostrar diagramas de consumidores, DLQ, retries y una demo en tiempo real.

En proyectos reales, agregar documentacion de decisiones, pruebas automatizadas y observabilidad antes de dividir el sistema en servicios.

Educacion

Los siguientes proyectos estan ordenados solo por su tematica. La guia no los clasifica como mejores o peores; cada uno permite practicar un conjunto diferente de problemas de backend.

Plataforma Educativa

Intermedio/Avanzado

1. Idea general

Portal donde estudiantes se inscriben en cursos, consumen contenido y siguen su progreso. Puede crecer desde CRUD de cursos hasta analytics, pagos y recomendaciones.

2. Como funcionaria?

```
Student -> catalog -> enrollment -> content -> progress -> assessment -> certificate.
```

3. Usuarios del sistema

STUDENT, INSTRUCTOR, ADMIN.

4. Funcionalidades principales

- Catalogo
- Inscripciones
- Lecciones
- Progreso
- Quizzes
- Certificados
- Favoritos
- Comentarios
- Busquedas
- Reportes de aprendizaje

5. Entidades principales

- User
- Course
- Lesson
- Enrollment
- Progress
- Quiz
- Question
- Attempt
- Certificate

6. API REST

- GET /api/courses
- POST /api/courses/{id}/enroll
- GET /api/courses/{id}/lessons
- POST /api/lessons/{id}/progress
- POST /api/quizzes/{id}/attempts
- GET /api/certificates

7. Arquitectura inicial

```
Flutter/Web -> REST -> Spring Boot modules -> PostgreSQL  
+--> object storage / events
```

8. Tecnologías recomendadas

Obligatorio

- Spring Boot
- JPA
- PostgreSQL

Recomendado

- Security
- object storage
- Redis
- Kafka
- Flutter

Avanzado

Search, analytics pipeline, recommendation service

9. Problemas tecnicos interesantes

- Autorizacion de contenido comprado.
- Progreso idempotente.
- Uploads y storage.
- Queries de catalogo.

10. Ejemplo de escenario real

El estudiante abre la misma leccion en dos dispositivos y ambos envian progreso. El sistema debe converger sin retroceder el estado.

11. Evolucion del proyecto

1. Courses
2. Enrollment
3. Progress
4. Security
5. Storage + search
6. Events + analytics
7. Recommendation/search services

12. Que aprenderia

REST

* * * * *

Security

* * * * *

JPA

* * * * *

Object storage

* * * * *

Caching

* * * * *

Events

* * * * *

Analytics

* * * * *

Flutter

* * * * *

13. Dificultad

Clasificacion tecnica: **Intermedio/Avanzado**. La etiqueta expresa el tipo de problemas de ingenieria presentes; no es un ranking entre proyectos.

14. Como convertirlo en un proyecto de portfolio

Incluir player, progreso, roles y diagrama de almacenamiento de contenido.

En proyectos reales, agregar documentacion de decisiones, pruebas automatizadas y observabilidad antes de dividir el sistema en servicios.

Sistema de Cursos Online

Avanzado

1. Idea general

Marketplace de cursos con instructores, ventas, reseñas y acceso por suscripcion o compra. Une e-commerce con education.

2. Como funcionaria?

Visitor -> course page -> checkout -> enrollment -> learning -> review -> instructor analytics.

3. Usuarios del sistema

STUDENT, INSTRUCTOR, ADMIN.

4. Funcionalidades principales

- Publicacion de cursos
- Modulos
- Videos
- Precios
- Checkout
- Cupones
- Inscripcion
- Reseñas
- Ingresos del instructor
- Analytics

5. Entidades principales

- Instructor
- Course
- Module
- Lesson
- Price
- Order
- Enrollment
- Review
- Coupon
- Payout

6. API REST

- POST /api/instructor/courses
- GET /api/courses/{id}
- POST /api/orders
- POST /api/enrollments
- POST /api/reviews
- GET /api/instructor/analytics

7. Arquitectura inicial

Catalog + Order + Learning modules -> events -> payout/analytics

8. Tecnologías recomendadas

Obligatorio

- Spring Boot
- JPA
- PostgreSQL
- Security

Recomendado

- Redis
- Kafka
- storage
- Docker
- Flutter

Avanzado

Search, payment provider, analytics service

9. Problemas tecnicos interesantes

- Entitlements: solo compradores pueden acceder.
- Reparto de ingresos.
- Versionado de cursos.
- Moderacion de contenido.

10. Ejemplo de escenario real

Un curso se actualiza mientras estudiantes estan viendo una leccion. El sistema puede mantener versiones de contenido para que las inscripciones existentes conserven referencias validas.

11. Evolucion del proyecto

1. Course CRUD
2. Learning
3. Checkout
4. Security
5. Events + analytics
6. Storage + search
7. Payout service

12. Que aprenderia

Security

* * * * *

JPA

* * * * *

Events

* * * * *

Storage

* * * * *

Search

* * * * *

Transactions

* * * * *

Architecture

* * * * *

Analytics

* * * * *

13. Dificultad

Clasificacion tecnica: **Avanzado**. La etiqueta expresa el tipo de problemas de ingenieria presentes; no es un ranking entre proyectos.

14. Como convertirlo en un proyecto de portfolio

Mostrar workflow del instructor, acceso protegido y calculo de revenue share.

En proyectos reales, agregar documentacion de decisiones, pruebas automatizadas y observabilidad antes de dividir el sistema en servicios.

LMS Empresarial

Avanzado

1. Idea general

Learning Management System para empresas con organizaciones, cursos obligatorios, cohortes, deadlines y reportes de compliance.

2. Como funcionaria?

HR -> assign course -> employee -> progress -> assessment -> certificate -> compliance report.

3. Usuarios del sistema

EMPLOYEE, MANAGER, HR_ADMIN, CONTENT_ADMIN.

4. Funcionalidades principales

- Organizaciones
- Asignacion de cursos
- Cohortes
- Deadlines
- Contenido obligatorio
- Quizzes
- Certificados
- Dashboards de manager
- Compliance reports
- Auditoria

5. Entidades principales

- Organization
- Employee
- Course
- Assignment
- Cohort
- Deadline
- Assessment
- Certificate
- ComplianceSnapshot

6. API REST

- POST /api/assignments
- GET /api/me/learning
- POST /api/assessments/{id}/attempts
- GET /api/teams/{id}/progress
- GET /api/reports/compliance
- POST /api/certificates

7. Arquitectura inicial

API -> Tenant/IAM module -> Learning modules -> PostgreSQL -> reporting projections

8. Tecnologías recomendadas

Obligatorio

- Spring Boot
- Security
- JPA
- PostgreSQL

Recomendado

- Redis
- Kafka
- scheduled jobs
- Docker

Avanzado

Multi-tenant DB strategy, analytics warehouse, SSO/OAuth2

9. Problemas técnicos interesantes

- Multi-tenancy.
- Deadlines y jobs.
- Reportes pesados.
- Permisos manager vs employee.

10. Ejemplo de escenario real

Un manager debe ver solo los empleados de su equipo. La autorización debe aplicarse sobre el alcance organizacional y no solamente por rol.

11. Evolución del proyecto

1. Single org
2. Tenant model
3. Assignments
4. Reports
5. Events + snapshots
6. SSO
7. Analytics service

12. Qué aprendería

Security

* * * * *

Multi-tenancy

* * * * *

Scheduling

* * * * *

JPA

* * * * *

Events

* * * * *

Reporting

* * * * *

Authorization

* * * * *

Architecture

* * * * *

13. Dificultad

Clasificación técnica: **Avanzado**. La etiqueta expresa el tipo de problemas de ingeniería presentes; no es un ranking entre proyectos.

14. Cómo convertirlo en un proyecto de portfolio

Documentar tenant isolation, RBAC/ABAC y reportes de compliance.

En proyectos reales, agregar documentación de decisiones, pruebas automatizadas y observabilidad antes de dividir el sistema en servicios.

Plataforma de Exámenes

Avanzado

1. Idea general

Sistema de evaluaciones con bancos de preguntas, intentos, temporizadores, correccion automatica y analiticas.

2. Como funcionaria?

```
Student -> exam start -> question delivery -> answers -> submit/timeout -> grading -> result -> analytics.
```

3. Usuarios del sistema

STUDENT, TEACHER, ADMIN.

4. Funcionalidades principales

- Question bank
- Exams
- Randomizacion
- Temporizador
- Intentos
- Autosave
- Correccion
- Rubricas
- Feedback
- Analytics

5. Entidades principales

- Exam
- Question
- QuestionOption
- Attempt
- Answer
- Grade
- Rubric
- ExamPolicy

6. API REST

- POST /api/exams/{id}/attempts
- GET /api/attempts/{id}
- PUT /api/attempts/{id}/answers
- POST /api/attempts/{id}/submit
- GET /api/results/{id}
- GET /api/analytics/exams/{id}

7. Arquitectura inicial

```
REST -> Exam service -> Attempt repository -> PostgreSQL
+--> Scheduler/timeout worker
```

8. Tecnologías recomendadas

Obligatorio

- Spring Boot
- JPA
- PostgreSQL

Recomendado

- Redis
- WebSockets
- Security
- scheduler
- Docker

Avanzado

Event-driven grading, analytics pipeline

9. Problemas tecnicos interesantes

- Autosave concurrente.
- Timeout exacto.
- Randomizacion reproducible.
- Integridad de intentos.

10. Ejemplo de escenario real

El estudiante envia una respuesta justo cuando expira el tiempo. El servidor necesita una regla determinista sobre timestamp del servidor y estado del intento.

11. Evolucion del proyecto

1. Question bank
2. Attempts
3. Autosave
4. Security
5. Real-time timer
6. Async grading
7. Analytics

12. Que aprenderia

Transactions

* * * * *

Scheduling

* * * * *

WebSockets

* * * * *

Security

* * * * *

JPA

* * * * *

Concurrency

* * * * *

Testing

* * * * *

Analytics

* * * * *

13. Dificultad

Clasificacion tecnica: **Avanzado**. La etiqueta expresa el tipo de problemas de ingenieria presentes; no es un ranking entre proyectos.

14. Como convertirlo en un proyecto de portfolio

Mostrar politica temporal, pruebas de timeout y resultados reproducibles.

En proyectos reales, agregar documentacion de decisiones, pruebas automatizadas y observabilidad antes de dividir el sistema en servicios.

Sistema de Gestion Universitaria

Avanzado

1. Idea general

Backoffice academico con alumnos, carreras, materias, inscripciones, correlatividades, cursadas y actas. Es excelente para modelado relacional y reglas complejas.

2. Como funcionaria?

```
Student -> academic plan -> enrollment -> prerequisites validation -> course -> grade -> transcript.
```

3. Usuarios del sistema

STUDENT, PROFESSOR, SECRETARY, ADMIN.

4. Funcionalidades principales

- Carreras
- Planes de estudio
- Materias
- Correlatividades
- Inscripciones
- Comisiones
- Asistencia
- Notas
- Actas
- Historia academica

5. Entidades principales

- Student
- Career
- StudyPlan
- Course
- Prerequisite
- Section
- Enrollment
- Attendance
- Grade
- Transcript

6. API REST

- GET /api/careers
- GET /api/study-plans/{id}
- POST /api/enrollments
- POST /api/grades
- GET /api/transcript
- GET /api/students/{id}/status

7. Arquitectura inicial

```
Controller -> Academic Service -> rule modules -> PostgreSQL
```

8. Tecnologías recomendadas

Obligatorio

- Spring Boot
- JPA
- PostgreSQL
- Security

Recomendado

- Flyway
- Testcontainers
- Redis
- Docker

Avanzado

Event-based academic reporting, external identity providers

9. Problemas técnicos interesantes

- Reglas de correlatividad.
- Consistencia de inscripciones.
- Historial inmutable.
- Consultas relacionales complejas.

10. Ejemplo de escenario real

Un alumno intenta inscribirse a una materia sin correlativas completas. El dominio debe validar el plan vigente, no una versión antigua del mismo.

11. Evolución del proyecto

1. CRUD académico
2. Rules
3. Security + roles
4. Audit
5. Events + reports
6. Performance
7. Modularization

12. Qué aprendería

Relational modeling

* * * * *

JPA

* * * * *

SQL

* * * * *

Security

* * * * *

Transactions

* * * * *

Domain rules

* * * * *

Testing

* * * * *

Reporting

* * * * *

13. Dificultad

Clasificación técnica: **Avanzado**. La etiqueta expresa el tipo de problemas de ingeniería presentes; no es un ranking entre proyectos.

14. Cómo convertirlo en un proyecto de portfolio

El modelo ER y las reglas de negocio deben estar bien documentados; son parte central del proyecto.

En proyectos reales, agregar documentación de decisiones, pruebas automatizadas y observabilidad antes de dividir el sistema en servicios.

Sistema de Seguimiento Academico

Avanzado

1. Idea general

Dashboard que integra calificaciones, asistencia, actividad y alertas de riesgo academico. Puede consumir eventos de otros sistemas.

2. Como funcionaria?

Academic sources -> ingestion -> normalized student timeline -> rules -> alerts -> dashboard.

3. Usuarios del sistema

STUDENT, ADVISOR, TEACHER, ADMIN.

4. Funcionalidades principales

- Timeline academica
- Asistencia
- Calificaciones
- Alertas
- Metas
- Comparativas historicas
- Notas de tutor
- Reportes
- Importaciones
- Notifications

5. Entidades principales

- Student
- AcademicEvent
- Attendance
- Grade
- AlertRule
- Alert
- Goal
- AdvisorNote
- ImportBatch

6. API REST

- GET /api/students/{id}/timeline
- GET /api/students/{id}/alerts
- POST /api/alerts/{id}/ack
- POST /api/imports
- GET /api/reports/cohorts
- GET /api/goals

7. Arquitectura inicial

Source systems -> Kafka/importers -> Academic timeline store -> REST dashboard

8. Tecnologías recomendadas

Obligatorio

- Spring Boot
- PostgreSQL
- Kafka

Recomendado

- Redis
- batch import
- Flutter
- Docker

Avanzado

Event streaming, data warehouse, recommendation/risk service

9. Problemas técnicos interesantes

- Normalización de fuentes heterogéneas.
- Eventos tardíos.
- Reportes históricos.
- Alertas duplicadas.

10. Ejemplo de escenario real

Una nota llega después de un evento de alerta. El sistema recalcula el estado sin perder la trazabilidad de que alerta se generó y por qué.

11. Evolución del proyecto

1. Imports
2. Timeline
3. Rules
4. Kafka
5. Dashboards
6. Analytics
7. Streaming architecture

12. Qué aprendería

Kafka

* * * * *

ETL-like ingestion

* * * * *

SQL

* * * * *

Events

* * * * *

Caching

* * * * *

Rules

* * * * *

Analytics

* * * * *

Observability

* * * * *

13. Dificultad

Clasificación técnica: **Avanzado**. La etiqueta expresa el tipo de problemas de ingeniería presentes; no es un ranking entre proyectos.

14. Cómo convertirlo en un proyecto de portfolio

Mostrar pipeline de ingestión, esquema de eventos y dashboard de alertas.

En proyectos reales, agregar documentación de decisiones, pruebas automatizadas y observabilidad antes de dividir el sistema en servicios.

IA y programacion

Los siguientes proyectos estan ordenados solo por su tematica. La guia no los clasifica como mejores o peores; cada uno permite practicar un conjunto diferente de problemas de backend.

AI Coding Platform

Muy Avanzado

1. Idea general

Plataforma donde un usuario crea repositorios, describe una tarea y recibe asistencia de IA sobre el código. Spring Boot gestiona usuarios, proyectos, jobs, permisos y persistencia; el modelo puede vivir como servicio separado.

2. Como funcionaria?

```
User -> project -> prompt/task -> Spring Boot orchestrator -> AI service -> tool execution -> patch/diff -> approval -> commit.
```

3. Usuarios del sistema

DEVELOPER, ADMIN, WORKSPACE_OWNER.

4. Funcionalidades principales

- Proyectos
- Repositorios
- Prompts
- Historial
- Diffs
- Aprobacion de cambios
- Context gathering
- Task queue
- Usage limits
- Audit log

5. Entidades principales

- User
- Workspace
- Project
- Repository
- AiTask
- Prompt
- ToolCall
- Patch
- Approval
- UsageRecord

6. API REST

- POST /api/ai/tasks
- GET /api/ai/tasks/{id}
- GET /api/projects/{id}/files
- POST /api/patches/{id}/approve
- GET /api/usage
- POST /api/repositories

7. Arquitectura inicial

```
Flutter/Web -> Spring Boot orchestrator -> AI Service -> tools/sandbox -> artifact store
+--> DB / event queue
```

8. Tecnologías recomendadas

Obligatorio

- Spring Boot
- PostgreSQL
- Security
- REST

Recomendado

- Redis
- Kafka
- object storage
- Docker
- Python model service

Avanzado

RAG, agents, sandbox orchestration, vector DB, microservices

9. Problemas tecnicos interesantes

- Context window y seleccion de archivos relevantes.
- Seguridad de ejecucion de herramientas.
- Jobs largos y timeouts.
- Idempotencia de tareas y consumo de tokens.

10. Ejemplo de escenario real

La IA genera un parche y un usuario lo ejecuta sin revisar. El sistema debe separar generacion, preview, aprobacion y ejecucion, dejando auditoria de cada paso.

11. Evolucion del proyecto

1. Prompt assistant
2. Project context
3. Tool calls
4. Async jobs
5. RAG
6. Sandbox
7. Agent orchestration

12. Que aprenderia

Spring orchestration *****	Queues *****	Security *****
AI integration *****	RAG *****	Docker *****
Async processing *****	Architecture *****	

13. Dificultad

Clasificacion tecnica: **Muy Avanzado** . La etiqueta expresa el tipo de problemas de ingenieria presentes; no es un ranking entre proyectos.

14. Como convertirlo en un proyecto de portfolio

Mostrar una demo end-to-end donde la IA propone un diff y el usuario lo aprueba; documentar el limite de confianza del agente.

En proyectos reales, agregar documentacion de decisiones, pruebas automatizadas y observabilidad antes de dividir el sistema en servicios.

Asistente de Programacion

Avanzado

1. Idea general

Asistente conversacional orientado a proyectos reales: explica errores, genera ejemplos y consulta documentacion interna. El backend mantiene contexto y controla limites.

2. Como funcionaria?

User -> conversation -> context retrieval -> AI provider -> streamed response -> persistence -> usage metering.

3. Usuarios del sistema

DEVELOPER, ADMIN.

4. Funcionalidades principales

- Conversaciones
- Contexto por proyecto
- Streaming
- Historial
- Snippets
- Citas de documentos
- Usage limits
- Feedback
- Prompt templates
- Moderation

5. Entidades principales

- User
- Conversation
- Message
- ProjectContext
- Document
- UsageRecord
- PromptTemplate
- Feedback

6. API REST

- POST /api/conversations
- POST /api/conversations/{id}/messages
- GET /api/conversations/{id}
- GET /api/projects/{id}/context
- GET /api/usage
- POST /api/feedback

7. Arquitectura inicial

Client -> Spring Boot -> context retrieval -> AI provider -> SSE/WebSocket -> DB

8. Tecnologías recomendadas

Obligatorio

- Spring Boot
- Web
- PostgreSQL
- Security

Recomendado

- Redis
- SSE/WebSockets
- vector DB
- Docker

Avanzado

RAG service, model gateway, semantic cache

9. Problemas tecnicos interesantes

- Streaming de tokens.
- Contexto excesivo.
- Cost/usage metering.
- Persistencia de conversaciones largas.

10. Ejemplo de escenario real

Una respuesta tarda mucho. El servidor envia tokens progresivamente y registra el estado del stream para que una desconexion no corrompa el historial.

11. Evolucion del proyecto

1. Chat
2. Project context
3. Streaming
4. Usage limits
5. RAG
6. Semantic cache
7. Model gateway

12. Que aprenderia

Spring WebFlux or SSE

Caching

Security

AI APIs

RAG

Persistence

Observability

Cost control

13. Dificultad

Clasificacion tecnica: **Avanzado**. La etiqueta expresa el tipo de problemas de ingenieria presentes; no es un ranking entre proyectos.

14. Como convertirlo en un proyecto de portfolio

Mostrar streaming, contexto de repositorio y un reporte de uso por proyecto.

En proyectos reales, agregar documentación de decisiones, pruebas automatizadas y observabilidad antes de dividir el sistema en servicios.

Plataforma de Ejecucion deCodigo

Muy Avanzado

1. Idea general

Servicio que recibe codigo, crea jobs de ejecucion aislados, aplica limites de CPU/memoria/tiempo y devuelve stdout/stderr. Es una introduccion fuerte a workers y sandboxing.

2. Como funcionaria?

```
User -> submit -> job queue -> worker -> sandbox -> result store -> status API -> stream logs.
```

3. Usuarios del sistema

USER, INSTRUCTOR, ADMIN.

4. Funcionalidades principales

- Subida de codigo
- Lenguajes
- Compilacion
- Tests
- Timeouts
- Logs
- Queues
- Retries
- Artefactos
- Historial de ejecuciones

5. Entidades principales

- Submission
- ExecutionJob
- TestCase
- ExecutionResult
- Artifact
- RuntimeConfig
- UsageQuota

6. API REST

- POST /api/submissions
- POST /api/submissions/{id}/run
- GET /api/runs/{id}
- GET /api/runs/{id}/logs
- POST /api/runs/{id}/cancel
- GET /api/runtimes

7. Arquitectura inicial

```
API -> Queue -> Runner worker -> isolated sandbox -> result store
                        ^                               |
                        +----- status/events -----+
```

8. Tecnologías recomendadas

Obligatorio

- Spring Boot
- PostgreSQL
- Docker
- queue

Recomendado

- Redis
- Kafka/RabbitMQ
- WebSockets
- object storage

Avanzado

Kubernetes jobs, sandbox runtime, autoscaling

9. Problemas técnicos interesantes

- Ejecución no confiable de código no confiable.
- Aislamiento.
- Resource limits.
- Colas y backpressure.
- Reintentos sin ejecutar dos veces de forma peligrosa.

10. Ejemplo de escenario real

Un usuario envía un loop infinito. El runner debe detener el proceso por timeout y liberar recursos sin afectar otros jobs.

11. Evolución del proyecto

1. Local runner
2. Queue
3. Docker isolation
4. Multiple languages
5. Scaling workers
6. Autoscaling
7. Kubernetes jobs

12. Qué aprendería

Queues

* * * * *

Docker

* * * * *

Concurrency

* * * * *

Resource control

* * * * *

WebSockets

* * * * *

Security

* * * * *

Distributed workers

* * * * *

Observability

* * * * *

13. Dificultad

Clasificación técnica: **Muy Avanzado**. La etiqueta expresa el tipo de problemas de ingeniería presentes; no es un ranking entre proyectos.

14. Como convertirlo en un proyecto de portfolio

El principal activo del portfolio es el sistema de jobs y sus limites; no conviene venderlo como un simple endpoint que ejecuta comandos.

En proyectos reales, agregar documentacion de decisiones, pruebas automatizadas y observabilidad antes de dividir el sistema en servicios.

RAG sobre Documentos

Muy Avanzado

1. Idea general

Aplicacion donde el usuario sube documentos y realiza preguntas basadas en su contenido. Spring Boot administra identidad, documentos, permisos, jobs de ingestion y consultas; otro servicio puede manejar embeddings.

2. Como funcionaria?

```
Upload -> object storage -> parsing/OCR -> chunking -> embeddings -> vector index -> retrieval -> LLM  
-> cited answer.
```

3. Usuarios del sistema

USER, WORKSPACE_ADMIN, PLATFORM_ADMIN.

4. Funcionalidades principales

- Upload
- Folders
- Permisos
- Versionado
- Ingestion jobs
- Chunking
- Embeddings
- Busqueda semantica
- Respuestas con fuentes
- Delete/reindex

5. Entidades principales

- Workspace
- Document
- DocumentVersion
- Chunk
- EmbeddingRef
- IngestionJob
- Query
- Citation
- Permission

6. API REST

- POST /api/documents
- GET /api/documents/{id}
- POST /api/documents/{id}/reindex
- POST /api/query
- GET /api/query/{id}
- DELETE /api/documents/{id}

7. Arquitectura inicial

```
Spring Boot -> document store -> async ingestion -> vector DB  
Query -> retrieval -> AI service
```

8. Tecnologías recomendadas

Obligatorio

- Spring Boot
- PostgreSQL
- object storage

Recomendado

- Kafka
- Redis
- vector DB
- Python embedding service
- Docker

Avanzado

Hybrid search, reranking, multi-tenant vector isolation

9. Problemas tecnicos interesantes

- Ingestion asincrona.
- Versionado e indexacion.
- Permisos en retrieval.
- Respuestas sin evidencia.

10. Ejemplo de escenario real

Se elimina un documento mientras corre su ingestion. El pipeline debe reconocer la version y evitar que fragmentos viejos permanezcan consultables.

11. Evolucion del proyecto

1. Upload
2. Parser
3. Vector search
4. Citations
5. Async jobs
6. Permission-aware retrieval
7. Scalable AI service

12. Que aprenderia

Async *****	Kafka *****	RAG *****
Vector search *****	Security *****	Storage *****
Eventual consistency *****	Observability *****	

13. Dificultad

Clasificacion tecnica: **Muy Avanzado** . La etiqueta expresa el tipo de problemas de ingenieria presentes; no es un ranking entre proyectos.

14. Como convertirlo en un proyecto de portfolio

La demo debe mostrar ingestion, fuentes citadas, permisos y reindexacion de una version.

En proyectos reales, agregar documentacion de decisiones, pruebas automatizadas y observabilidad antes de dividir el sistema en servicios.

Plataforma de Agentes IA

Muy Avanzado

1. Idea general

Sistema para definir agentes que ejecutan herramientas y workflows bajo politicas. Spring Boot se ocupa de usuarios, configuracion, ejecuciones, auditoria y colas; el runtime de agente puede ser independiente.

2. Como funcionaria?

```
User -> agent config -> task -> planner/runtime -> tool calls -> approvals -> result -> audit/event log.
```

3. Usuarios del sistema

USER, WORKSPACE_ADMIN, PLATFORM_ADMIN.

4. Funcionalidades principales

- Definicion de agentes
- Tools
- Policies
- Task queue
- Execution trace
- Human approval
- Secrets references
- Usage limits
- Retry policies
- Run history

5. Entidades principales

- Agent
- Tool
- Policy
- AgentRun
- ToolCall
- Approval
- SecretRef
- UsageRecord
- RunEvent

6. API REST

- POST /api/agents
- POST /api/agents/{id}/runs
- GET /api/runs/{id}
- POST /api/approvals/{id}
- GET /api/runs/{id}/events
- GET /api/usage

7. Arquitectura inicial

```
Spring Boot control plane -> queue -> agent runtime -> tools
+--> event store / audit / approvals
```

8. Tecnologías recomendadas

Obligatorio

- Spring Boot
- PostgreSQL
- Security

Recomendado

- Kafka
- Redis
- Docker
- Python agent runtime

Avanzado

Workflow engine, Kubernetes, policy service, sandboxing

9. Problemas técnicos interesantes

- Tool permissions.
- Long-running workflows.
- Human-in-the-loop.
- State persistence y retries.

10. Ejemplo de escenario real

Un agente intenta utilizar una herramienta fuera de su policy. El control plane rechaza la llamada antes de que llegue al runtime.

11. Evolucion del proyecto

1. Single agent
2. Tool registry
3. Async runs
4. Approvals
5. Policies
6. Scaling
7. Workflow platform

12. Que aprenderia

Distributed workflows

* * * * *

Security

* * * * *

Kafka

* * * * *

Persistence

* * * * *

AI orchestration

* * * * *

Docker

* * * * *

Observability

* * * * *

Architecture

* * * * *

13. Dificultad

Clasificación técnica: **Muy Avanzado**. La etiqueta expresa el tipo de problemas de ingeniería presentes; no es un ranking entre proyectos.

14. Como convertirlo en un proyecto de portfolio

Mostrar un execution trace completo con tool calls, policy checks y aprobaciones humanas.

En proyectos reales, agregar documentación de decisiones, pruebas automatizadas y observabilidad antes de dividir el sistema en servicios.

AI Code Review Platform

Avanzado

1. Idea general

Servicio que analiza pull requests, identifica riesgos, resume cambios y deja comentarios. Puede integrar GitHub/ GitLab y combinar reglas estaticas con IA.

2. Como funcionaria?

```
Webhook PR -> fetch diff -> static checks -> AI review -> findings -> comment proposal -> human approval -> publish.
```

3. Usuarios del sistema

DEVELOPER, TEAM_ADMIN, PLATFORM_ADMIN.

4. Funcionalidades principales

- Webhooks
- Diff ingestion
- Rule checks
- AI review
- Severity categories
- Comments
- Approval workflow
- Repository settings
- Usage limits
- Review history

5. Entidades principales

- Repository
- PullRequest
- Diff
- ReviewRun
- Finding
- Rule
- CommentDraft
- ReviewPolicy
- UsageRecord

6. API REST

- POST /api/webhooks/git
- POST /api/reviews
- GET /api/reviews/{id}
- POST /api/findings/{id}/resolve
- PATCH /api/review-policies
- GET /api/repositories

7. Arquitectura inicial

```
Git provider -> webhook -> Spring Boot -> review job -> static tools + AI -> findings -> provider API
```

8. Tecnologías recomendadas

Obligatorio

- Spring Boot
- PostgreSQL
- REST integrations

Recomendado

- Kafka
- Redis
- Docker
- Python analysis service

Avanzado

RAG over codebase, agentic remediation, separate workers

9. Problemas tecnicos interesantes

- Idempotencia de webhooks.
- Diffs grandes.
- Cost de IA.
- Falsos positivos.
- Aprobacion humana antes de publicar comentarios.

10. Ejemplo de escenario real

GitHub reenvia un webhook. El sistema detecta el mismo delivery id y devuelve el estado existente en vez de lanzar dos reviews.

11. Evolucion del proyecto

1. Manual review job
2. Webhook
3. Async queue
4. Static analyzers
5. AI
6. RAG
7. Scalable workers

12. Que aprenderia

Webhooks

Async

AI integration

Security

Git APIs

Kafka

Observability

Testing

13. Dificultad

Clasificacion tecnica: **Avanzado**. La etiqueta expresa el tipo de problemas de ingenieria presentes; no es un ranking entre proyectos.

14. Como convertirlo en un proyecto de portfolio

Mostrar una PR real de ejemplo con findings y trazabilidad desde webhook hasta comentario.

En proyectos reales, agregar documentacion de decisiones, pruebas automatizadas y observabilidad antes de dividir el sistema en servicios.

Sistemas de gran escala

Los siguientes proyectos estan ordenados solo por su tematica. La guia no los clasifica como mejores o peores; cada uno permite practicar un conjunto diferente de problemas de backend.

Uber Simplificado

Muy Avanzado

1. Idea general

Plataforma de viajes con pasajeros, conductores, matching, tracking y pricing simple. Es un laboratorio de geolocalización, eventos y estado.

2. Como funcionaria?

Rider -> request -> dispatch -> driver assignment -> trip -> GPS updates -> completion -> rating.

3. Usuarios del sistema

RIDER, DRIVER, OPS ADMIN.

4. Funcionalidades principales

- Requests
- Driver availability
- Matching
- Trip lifecycle
- GPS pings
- ETA
- Pricing estimate
- Cancellation
- Rating
- Operations dashboard

5. Entidades principales

- Rider
- Driver
- Vehicle
- TripRequest
- Trip
- LocationPing
- Assignment
- Fare
- Rating

6. API REST

- POST /api/trips/requests
- POST /api/drivers/{id}/availability
- POST /api/drivers/{id}/location
- GET /api/trips/{id}
- POST /api/trips/{id}/cancel
- WS /ws/trips/{id}

7. Arquitectura inicial

Mobile -> API Gateway -> Trip/Dispatch modules -> Redis Geo + PostgreSQL
+> Kafka events -> analytics

8. Tecnologías recomendadas

Obligatorio

- Spring Boot
- PostgreSQL
- Redis

Recomendado

- WebSockets
- Kafka
- Docker
- Flutter

Avanzado

Microservices, partitioning, Kubernetes, tracing

9. Problemas tecnicos interesantes

- Matching bajo latencia.
- Ubicaciones frecuentes.
- Driver assignment race conditions.
- Eventual consistency.

10. Ejemplo de escenario real

Dos servidores intentan asignar el mismo conductor. Una estrategia de reserva atomica o versionado evita doble asignacion.

11. Evolucion del proyecto

1. Trip MVP
2. Driver app
3. Matching
4. Redis Geo
5. WebSockets
6. Kafka + scaling
7. Separate dispatch service

12. Que aprenderia

Redis

* * * * *

WebSockets

* * * * *

Kafka

* * * * *

Concurrency

* * * * *

Geospatial

* * * * *

Scaling

* * * * *

Microservices

* * * * *

Observability

* * * * *

13. Dificultad

Clasificacion tecnica: **Muy Avanzado**. La etiqueta expresa el tipo de problemas de ingenieria presentes; no es un ranking entre proyectos.

14. Como convertirlo en un proyecto de portfolio

Mostrar matching, mapa en tiempo real y una explicacion de particionado por region.

En proyectos reales, agregar documentación de decisiones, pruebas automatizadas y observabilidad antes de dividir el sistema en servicios.

Airbnb Simplificado

Avanzado

1. Idea general

Marketplace de alojamientos con calendarios, disponibilidad, reservas, pagos simulados y reviews. El eje tecnico es evitar dobles reservas y manejar reglas temporales.

2. Como funcionaria?

Guest -> search -> property -> availability -> hold -> booking -> payment -> stay -> review.

3. Usuarios del sistema

GUEST, HOST, ADMIN.

4. Funcionalidades principales

- Listings
- Calendars
- Availability
- Search
- Holds
- Bookings
- Payments
- Cancellations
- Reviews
- Host dashboard

5. Entidades principales

- Host
- Property
- Unit
- Availability
- Booking
- Hold
- Guest
- Payment
- Review
- CancellationPolicy

6. API REST

- GET /api/listings
- GET /api/listings/{id}/availability
- POST /api/holds
- POST /api/bookings
- POST /api/bookings/{id}/cancel
- GET /api/host/dashboard

7. Arquitectura inicial

Search -> Availability -> Booking transaction -> Payment -> Events
Redis cache + search index

8. Tecnologías recomendadas

Obligatorio

- Spring Boot
- PostgreSQL
- JPA

Recomendado

- Redis
- search engine
- Kafka
- Security
- Docker

Avanzado

Separate search/booking services, geo search, Kubernetes

9. Problemas tecnicos interesantes

- Overbooking.
- Search freshness.
- Timezone.
- Cancellation policies.
- Distributed payment/booking state.

10. Ejemplo de escenario real

Un host modifica disponibilidad mientras un guest mantiene un hold. La confirmacion debe revalidar la version vigente antes del commit.

11. Evolucion del proyecto

1. Listings
2. Booking
3. Hold locking
4. Search
5. Events
6. Scale search
7. Service separation

12. Que aprenderia

JPA

* * * * *

Concurrency

* * * * *

Redis

* * * * *

Search

* * * * *

Kafka

* * * * *

Security

* * * * *

Transactions

* * * * *

Scalability

* * * * *

13. Dificultad

Clasificacion tecnica: **Avanzado**. La etiqueta expresa el tipo de problemas de ingenieria presentes; no es un ranking entre proyectos.

14. Como convertirlo en un proyecto de portfolio

La prueba clave es la prevencion de doble reserva y el diseño de disponibilidad.

En proyectos reales, agregar documentacion de decisiones, pruebas automatizadas y observabilidad antes de dividir el sistema en servicios.

Sistema tipo Netflix

Avanzado

1. Idea general

Plataforma de contenido con catalogo, perfiles, reproduccion y recomendaciones simples. El objetivo es aprender CDN-like storage abstraction, metadata, event ingestion y personalization.

2. Como funcionaria?

User -> browse -> play -> playback event -> progress -> recommendation -> continue watching.

3. Usuarios del sistema

VIEWER, CONTENT_EDITOR, ADMIN.

4. Funcionalidades principales

- Catalogo
- Profiles
- Continue watching
- Watch history
- Playback sessions
- Search
- Genres
- Recommendations
- Content metadata
- Analytics

5. Entidades principales

- User
- Profile
- Title
- Episode
- Genre
- PlaybackSession
- WatchEvent
- WatchProgress
- Recommendation

6. API REST

- GET /api/catalog
- GET /api/titles/{id}
- POST /api/playback/start
- POST /api/playback/progress
- GET /api/recommendations
- GET /api/profiles/{id}/history

7. Arquitectura inicial

Client -> Spring Boot -> Catalog/Playback -> PostgreSQL
Events -> Kafka -> Recommendation/analytics

8. Tecnologías recomendadas

Obligatorio

- Spring Boot
- JPA
- PostgreSQL

Recomendado

- Redis
- Kafka
- object storage
- CDN concepts
- Docker

Avanzado

Search cluster, recommendation service, event streaming

9. Problemas tecnicos interesantes

- High read volume.
- Progress updates frecuentes.
- Event ingestion.
- Cache consistency.

10. Ejemplo de escenario real

Miles de clientes envian progress updates cada pocos segundos. En lugar de escribir todo sin control, se agrupan eventos o se limita la frecuencia.

11. Evolucion del proyecto

1. Catalog
2. Playback state
3. Redis
4. Kafka
5. Search
6. Recommendations
7. Scale read paths

12. Que aprenderia

Caching

* * * * *

Kafka

* * * * *

Event ingestion

* * * * *

REST

* * * * *

Performance

* * * * *

Search

* * * * *

Distributed systems

* * * * *

Analytics

* * * * *

13. Dificultad

Clasificacion tecnica: **Avanzado**. La etiqueta expresa el tipo de problemas de ingenieria presentes; no es un ranking entre proyectos.

14. Como convertirlo en un proyecto de portfolio

Mostrar eventos de reproduccion, cache strategy y un recommendation MVP basado en historial.

En proyectos reales, agregar documentacion de decisiones, pruebas automatizadas y observabilidad antes de dividir el sistema en servicios.

Sistema tipo Spotify

Avanzado

1. Idea general

Servicio de musica con biblioteca, playlists, seguimiento de reproduccion y recomendaciones. El backend practica relaciones, busqueda y eventos de uso.

2. Como funcionaria?

```
User -> search -> track -> playback -> event -> history -> playlist/recommendation.
```

3. Usuarios del sistema

LISTENER, ARTIST, ADMIN.

4. Funcionalidades principales

- Artists
- Albums
- Tracks
- Search
- Playlists
- Likes
- Playback history
- Following
- Recommendations
- Artist analytics

5. Entidades principales

- Artist
- Album
- Track
- User
- Playlist
- PlaylistItem
- Like
- PlaybackEvent
- Follow

6. API REST

- GET /api/search
- GET /api/tracks/{id}
- POST /api/playlists
- POST /api/playlists/{id}/items
- POST /api/playback-events
- GET /api/recommendations

7. Arquitectura inicial

```
Client -> API -> catalog/search -> PostgreSQL/Redis
      playback events -> Kafka -> analytics/recommendation
```

8. Tecnologías recomendadas

Obligatorio

- Spring Boot
- PostgreSQL
- JPA

Recomendado

- Redis
- Kafka
- search
- object storage

Avanzado

Recommendation service, event streaming, CDN

9. Problemas técnicos interesantes

- Search.
- Hot tracks cache.
- Event volume.
- Playlist edits concurrentes.

10. Ejemplo de escenario real

Dos dispositivos modifican una playlist al mismo tiempo. Un version field y manejo de conflicto evita perder cambios silenciosamente.

11. Evolucion del proyecto

1. Catalog
2. Playlists
3. Search
4. Redis
5. Kafka
6. Recommendation
7. Read scaling

12. Que aprenderia

JPA

* * * * *

Search

* * * * *

Caching

* * * * *

Kafka

* * * * *

Concurrency

* * * * *

Events

* * * * *

Performance

* * * * *

Architecture

* * * * *

13. Dificultad

Clasificación técnica: **Avanzado**. La etiqueta expresa el tipo de problemas de ingeniería presentes; no es un ranking entre proyectos.

14. Como convertirlo en un proyecto de portfolio

La demo ideal es una playlist + historial + recommendation pipeline.

En proyectos reales, agregar documentación de decisiones, pruebas automatizadas y observabilidad antes de dividir el sistema en servicios.

Plataforma tipo Steam

Avanzado

1. Idea general

Tienda y biblioteca de videojuegos con cuentas, wishlist, compras, licencias, downloads metadata y reviews. Puede crecer hacia social y achievements.

2. Como funcionaria?

```
User -> store -> cart -> checkout -> entitlement -> library -> download metadata -> achievements/  
reviews.
```

3. Usuarios del sistema

PLAYER, DEVELOPER, MODERATOR, ADMIN.

4. Funcionalidades principales

- Store catalog
- Bundles
- Wishlist
- Cart
- Checkout
- Entitlements
- Library
- Reviews
- Achievements
- Developer dashboard

5. Entidades principales

- Game
- Developer
- Package
- Price
- Cart
- Order
- Entitlement
- LibraryItem
- Review
- Achievement
- UserProgress

6. API REST

- GET /api/store/games
- POST /api/cart/items
- POST /api/orders
- GET /api/library
- POST /api/reviews
- GET /api/achievements

7. Arquitectura inicial

```
Catalog -> Cart -> Order -> Entitlement -> Library  
Events -> achievements/reviews/notifications
```

8. Tecnologías recomendadas

Obligatorio

- Spring Boot
- JPA
- PostgreSQL

Recomendado

- Redis
- Kafka
- search
- storage
- Security
- Docker

Avanzado

CDN, license service, analytics pipeline

9. Problemas técnicos interesantes

- Entitlements idempotentes.
- Bundles y precios.
- Biblioteca derivada.
- Alta carga de catálogo.

10. Ejemplo de escenario real

El pago se confirma dos veces por webhook. Solo debe existir una licencia activa para el juego.

11. Evolucion del proyecto

1. Store
2. Cart
3. Entitlements
4. Search
5. Events
6. Reviews/achievements
7. Service extraction

12. Que aprenderia

Transactions

* * * * *

Events

* * * * *

Security

* * * * *

Search

* * * * *

Caching

* * * * *

Domain modeling

* * * * *

Async

* * * * *

Testing

* * * * *

13. Dificultad

Clasificación técnica: **Avanzado**. La etiqueta expresa el tipo de problemas de ingeniería presentes; no es un ranking entre proyectos.

14. Como convertirlo en un proyecto de portfolio

Demostrar entitlement flow, compras y event-driven achievements.

En proyectos reales, agregar documentacion de decisiones, pruebas automatizadas y observabilidad antes de dividir el sistema en servicios.

Mini AWS

Muy Avanzado

1. Idea general

Plataforma de cloud simplificada donde usuarios crean recursos ficticios: buckets, compute jobs, secrets y deployments. El objetivo es estudiar control planes, jobs y aislamiento logico.

2. Como funcionaria?

User -> API -> resource controller -> provisioning queue -> worker -> state store -> status events.

3. Usuarios del sistema

TENANT_USER, TENANT_ADMIN, PLATFORM_ADMIN.

4. Funcionalidades principales

- Projects
- Buckets
- Jobs
- Deployments
- Secrets metadata
- Quotas
- Resource states
- Activity log
- Usage metrics
- API tokens

5. Entidades principales

- Tenant
- Project
- Resource
- Job
- Deployment
- SecretRef
- Quota
- UsageMetric
- ApiToken

6. API REST

- POST /api/projects
- POST /api/jobs
- GET /api/jobs/{id}
- POST /api/deployments
- GET /api/resources
- GET /api/usage

7. Arquitectura inicial

CLI/Web -> Spring Boot Control Plane -> Queue -> Provisioner workers -> State store
+> Events

8. Tecnologías recomendadas

Obligatorio

- Spring Boot
- PostgreSQL
- Docker

Recomendado

- Kafka/RabbitMQ
- Redis
- Security
- OpenAPI

Avanzado

Kubernetes, Terraform provider concepts, service mesh

9. Problemas técnicos interesantes

- Estados asincronos.
- Quotas.
- Retries de provisioning.
- Idempotencia de resource creation.

10. Ejemplo de escenario real

El cliente pierde conexión durante la creación de un recurso. La siguiente llamada debe consultar el resource request existente en lugar de crear otro.

11. Evolución del proyecto

1. Resources CRUD
2. Async provisioning
3. Quotas
4. Workers
5. Events
6. Kubernetes
7. Multi-service control plane

12. Qué aprendería

Async jobs

* * * * *

Kafka/RabbitMQ

* * * * *

Docker

* * * * *

Security

* * * * *

State machines

* * * * *

Idempotency

* * * * *

Distributed systems

* * * * *

DevOps

* * * * *

13. Dificultad

Clasificación técnica: **Muy Avanzado**. La etiqueta expresa el tipo de problemas de ingeniería presentes; no es un ranking entre proyectos.

14. Cómo convertirlo en un proyecto de portfolio

El README debe explicar control plane vs data plane y mostrar estados de recursos.

En proyectos reales, agregar documentación de decisiones, pruebas automatizadas y observabilidad antes de dividir el sistema en servicios.

Sistema de Observabilidad

Muy Avanzado

1. Idea general

Plataforma que ingiere logs, metricas y trazas de aplicaciones para buscar incidencias y construir dashboards. Puede empezar almacenando eventos y evolucionar hacia pipelines distribuidos.

2. Como funcionaria?

```
Application -> collector/API -> ingestion queue -> processors -> storage -> query API -> dashboard/alerts.
```

3. Usuarios del sistema

ENGINEER, SRE/OPS, ADMIN.

4. Funcionalidades principales

- Log ingestion
- Metrics ingestion
- Trace intake
- Search
- Dashboards
- Alerts
- Retention rules
- Teams
- Annotations
- Incident timeline

5. Entidades principales

- Project
- Service
- LogEvent
- MetricPoint
- Trace
- Span
- AlertRule
- Alert
- Dashboard

6. API REST

- POST /api/logs
- POST /api/metrics
- POST /api/traces
- GET /api/logs/search
- POST /api/alerts
- GET /api/dashboards/{id}

7. Arquitectura inicial

```
Collectors -> Kafka -> ingestion workers -> storage indexes
Query API -> caches -> dashboard
```

8. Tecnologías recomendadas

Obligatorio

- Spring Boot
- PostgreSQL for control plane

Recomendado

- Kafka
- Redis
- OpenTelemetry
- Docker

Avanzado

ClickHouse/Elasticsearch, Kubernetes, distributed tracing

9. Problemas tecnicos interesantes

- Alta tasa de ingestion.
- Indexacion.
- Retention.
- Cardinality de metricas.
- Query performance.

10. Ejemplo de escenario real

Un servicio entra en loop y emite millones de logs. El sistema de ingestion aplica cuotas y backpressure para proteger el almacenamiento.

11. Evolucion del proyecto

1. Control plane
2. Logs
3. Metrics
4. Kafka ingestion
5. Search
6. Alerts
7. Scale storage/query

12. Que aprenderia

Kafka

* * * * *

Observability

* * * * *

Backpressure

* * * * *

Search

* * * * *

Caching

* * * * *

Performance

* * * * *

Distributed systems

* * * * *

DevOps

* * * * *

13. Dificultad

Clasificacion tecnica: **Muy Avanzado**. La etiqueta expresa el tipo de problemas de ingenieria presentes; no es un ranking entre proyectos.

14. Como convertirlo en un proyecto de portfolio

Un dashboard de observabilidad del propio sistema es especialmente valioso.

En proyectos reales, agregar documentacion de decisiones, pruebas automatizadas y observabilidad antes de dividir el sistema en servicios.

Plataforma de Deployment

Muy Avanzado

1. Idea general

Servicio donde un desarrollador registra un repo, configura un pipeline y lanza deployments a entornos de prueba. Es un proyecto completo de DevOps aplicado a backend.

2. Como funcionaria?

```
Git push -> webhook -> pipeline job -> build/test -> artifact -> deploy worker -> health check -> release state.
```

3. Usuarios del sistema

DEVELOPER, TEAM_ADMIN, PLATFORM_ADMIN.

4. Funcionalidades principales

- Repositories
- Environments
- Pipelines
- Builds
- Artifacts
- Deployments
- Rollbacks
- Secrets references
- Health checks
- Audit logs

5. Entidades principales

- Repository
- Environment
- Pipeline
- PipelineRun
- Artifact
- Deployment
- Release
- HealthCheck
- SecretRef

6. API REST

- POST /api/pipelines
- POST /api/webhooks/git
- POST /api/pipelines/{id}/runs
- GET /api/runs/{id}
- POST /api/deployments/{id}/rollback
- GET /api/environments

7. Arquitectura inicial

```
Git provider -> webhook -> Spring Boot control plane -> queue -> workers -> runtime  
+ -> deployment events
```

8. Tecnologías recomendadas

Obligatorio

- Spring Boot
- PostgreSQL
- Docker

Recomendado

- Kafka/RabbitMQ
- Redis
- Security
- Git APIs

Avanzado

Kubernetes, Terraform, OpenTelemetry, autoscaling

9. Problemas técnicos interesantes

- Long-running jobs.
- Retry seguro.
- Secrets.
- Rollback.
- Concurrent deployments al mismo entorno.

10. Ejemplo de escenario real

Dos pipelines quieren desplegar al mismo entorno. El sistema necesita una política de concurrencia, locks o cola por environment.

11. Evolucion del proyecto

1. Manual deploy
2. Pipeline
3. Async workers
4. Artifacts
5. Rollback
6. Kubernetes
7. Distributed deployment platform

12. Que aprenderia

Docker

* * * * *

Queues

* * * * *

CI/CD

* * * * *

Security

* * * * *

Concurrency

* * * * *

State machines

* * * * *

Kubernetes concepts

* * * * *

Observability

* * * * *

13. Dificultad

Clasificación técnica: **Muy Avanzado**. La etiqueta expresa el tipo de problemas de ingeniería presentes; no es un ranking entre proyectos.

14. Como convertirlo en un proyecto de portfolio

Mostrar un pipeline real con build, test, artifact, deploy simulado y rollback.

En proyectos reales, agregar documentacion de decisiones, pruebas automatizadas y observabilidad antes de dividir el sistema en servicios.

Sistema Global de Pagos

Muy Avanzado

1. Idea general

Arquitectura distribuida de laboratorio para enrutar pagos entre varios proveedores, monedas y regiones con idempotencia, retries y conciliación. Debe tratarse como simulador educativo, no como infraestructura financiera real.

2. Como funcionaria?

```
Merchant -> payment router -> provider selection -> provider adapter -> async callback -> ledger -> settlement/reconciliation.
```

3. Usuarios del sistema

MERCHANT, OPERATIONS, ADMIN.

4. Funcionalidades principales

- Multi-provider routing
- Currencies
- Payment states
- Retries
- Webhooks
- Idempotency
- Routing rules
- Settlement files
- Reconciliation
- Audit

5. Entidades principales

- Merchant
- Payment
- Provider
- PaymentAttempt
- RoutingRule
- WebhookEvent
- Settlement
- ReconciliationRecord
- AuditLog

6. API REST

- POST /api/payments
- GET /api/payments/{id}
- POST /api/webhooks/{provider}
- GET /api/settlements
- POST /api/reconciliation/run
- GET /api/merchants/{id}/stats

7. Arquitectura inicial

```
API -> Router -> Provider adapters -> Event bus -> Ledger/Settlement/Reconciliation
```

8. Tecnologías recomendadas

Obligatorio

- Spring Boot
- PostgreSQL
- Security

Recomendado

- Kafka
- Redis
- resilience patterns
- Docker
- OpenTelemetry

Avanzado

Regional services, multi-region data, workflow engine

9. Problemas tecnicos interesantes

- Provider outages.
- Duplicate callbacks.
- Event ordering.
- Cross-system reconciliation.
- Regional latency.

10. Ejemplo de escenario real

Un proveedor queda fuera de servicio mientras llegan pagos. El router deriva nuevas operaciones a otro proveedor y conserva trazabilidad de los intentos previos.

11. Evolucion del proyecto

1. Single provider
2. Provider adapter
3. Retries
4. Kafka + reconciliation
5. Multiple currencies
6. Regionalization
7. Service separation

12. Que aprenderia

Distributed systems

* * * * *

Kafka

* * * *

Resilience

* * * *

Transactions

* * * * *

Security

* * * *

Observability

* * * *

Idempotency

* * * * *

Architecture

* * * *

13. Dificultad

Clasificación técnica: **Muy Avanzado**. La etiqueta expresa el tipo de problemas de ingeniería presentes; no es un ranking entre proyectos.

14. Como convertirlo en un proyecto de portfolio

La parte mas importante es documentar estados, reconciliacion y fallos simulados.

En proyectos reales, agregar documentacion de decisiones, pruebas automatizadas y observabilidad antes de dividir el sistema en servicios.

Sistema de Event Streaming

Avanzado

1. Idea general

Plataforma educativa donde otros servicios publican eventos y diferentes consumidores crean proyecciones, alertas y analytics. Es un proyecto dedicado a comprender arquitectura event-driven.

2. Como funcionaria?

```
Producer service -> Kafka -> consumer groups -> processors -> materialized views -> query APIs.
```

3. Usuarios del sistema

DEVELOPER, PLATFORM_ADMIN.

4. Funcionalidades principales

- Event schemas
- Topics
- Consumer groups
- Replay
- Dead-letter queues
- Materialized views
- Metrics
- Replay UI
- Retention config
- Audit

5. Entidades principales

- EventEnvelope
- TopicConfig
- ConsumerGroup
- Projection
- ProcessingError
- ReplayJob
- MetricSample

6. API REST

- POST /api/topics
- POST /api/events/publish
- GET /api/consumer-groups
- POST /api/replays
- GET /api/projections/{name}
- GET /api/errors

7. Arquitectura inicial

```
Producers -> Kafka -> consumers -> projection stores  
+ -> DLQ / replay workers
```

8. Tecnologías recomendadas

Obligatorio

- Spring Boot
- Kafka
- PostgreSQL

Recomendado

- Schema registry
- Redis
- Docker
- OpenTelemetry

Avanzado

Kafka Streams, Kubernetes, multi-cluster concepts

9. Problemas técnicos interesantes

- Schema evolution.
- Replay sin duplicar efectos.
- Consumer lag.
- Poison messages.
- Exactly-once vs at-least-once tradeoffs.

10. Ejemplo de escenario real

Un consumer falla después de persistir el resultado pero antes de confirmar el offset. Al reintentar, el consumidor debe tolerar el duplicado mediante idempotencia o transacciones apropiadas.

11. Evolución del proyecto

1. Publish/consume
2. Schemas
3. DLQ
4. Replay
5. Materialized views
6. Kafka Streams
7. Cluster scaling

12. Qué aprendería

Kafka

* * * * *

Event-driven architecture

* * * * *

Idempotency

* * * * *

Schemas

* * * * *

Observability

* * * * *

Concurrency

* * * * *

Distributed systems

* * * * *

Testing

* * * * *

13. Dificultad

Clasificación técnica: **Avanzado**. La etiqueta expresa el tipo de problemas de ingeniería presentes; no es un ranking entre proyectos.

14. Como convertirlo en un proyecto de portfolio

Es especialmente util como proyecto de infraestructura para demostrar que entiendes eventos mas alla de usar Kafka en un CRUD.

En proyectos reales, agregar documentacion de decisiones, pruebas automatizadas y observabilidad antes de dividir el sistema en servicios.

Parte 3 - Proyectos que permiten aprender mas Spring Boot

Esta seleccion no es un ranking. Son ocho proyectos que concentran distintas capacidades de Spring y permiten estudiar por capas.

Proyecto	Spring / backend que aparece
Banco Digital	Spring Web, Dependency Injection, Data JPA, Transactions, Security, Validation, Kafka, Redis, Testing, Docker
Marketplace	Web, JPA, Security, transactions, cache, events, search, integrations
Plataforma de Chat	WebSockets, Security, async processing, Redis, event messaging
LMS Empresarial	Security, multi-tenancy, JPA, scheduling, authorization, reporting
RAG sobre Documentos	REST, Security, async jobs, storage integration, Kafka, service separation
Plataforma de Ejecucion deCodigo	Workers, queues, persistence, Docker orchestration, WebSockets
Plataforma de Deployment	REST, webhooks, queues, CI/CD integrations, Docker, observability
Sistema de Observabilidad	Kafka ingestion, query APIs, backpressure, OpenTelemetry, metrics

Metodo de estudio: cuando aparezca una nueva tecnologia, implementa primero la ruta feliz en el monolito. Despues fuerza fallos, reintentos y concurrencia. La segunda etapa suele enseñar mas que el CRUD inicial.

Parte 4 - Mapa de aprendizaje

Usa esta tabla para elegir un concepto y ubicar proyectos donde aparece de manera natural.

Concepto	Proyectos donde practicarlo
Controllers	Banco Digital, Tienda Online, Plataforma Educativa
Dependency Injection	Cualquier monolito Spring; especialmente Banco Digital y Marketplace
Services	Todos los proyectos del catalogo
Repositories / JPA	Banco Digital, Sistema Universitario, Inventario
Transactions	Banco Digital, Pagos, Suscripciones, Reservas
Security / JWT	Banco Digital, Marketplace, LMS, Chat
Redis	Delivery, Chat, Trading, Observabilidad
Kafka	Antifraude, Notificaciones, Netflix-like, Event Streaming
WebSockets	Chat, Discord Simplificado, Delivery, Trading
Docker	Todos los proyectos avanzados; runner de codigo y deployment especialmente
Microservices	Payments, Marketplace, Delivery, RAG, Deployment
Concurrency	Banco Digital, Exchange, Reservas, Inventario
Testing	Todos; usar Testcontainers e integracion en proyectos con DB
Observability	Sistema de Observabilidad, Pagos, Event Streaming, Deployment

Parte 5 - Ejemplo de evolucion: Banco Digital

```
Mes 1  Spring Boot + REST + PostgreSQL
Mes 2  JPA + Validation + Security
Mes 3  JWT + Roles + Testing
Mes 4  Redis + WebSockets
Mes 5  Kafka + eventos
Mes 6  Docker + CI/CD
Mes 7  Separacion por modulos
Mes 8  Microservicios selectivos
Mes 9  Observability
Mes 10 Load testing + optimization
```

Meses 1-3

Aprendes el modelo de programacion de Spring: controllers, services, repositories, dependency injection, persistence, validation y seguridad.

Meses 4-6

Aprendes que cambia cuando el sistema recibe mas lecturas, eventos y procesos asincronos. Redis, Kafka y Docker introducen nuevos modos de fallo.

Meses 7-8

Identificas limites de dominio y extraes solo los componentes que realmente tienen autonomia, carga o ciclo de despliegue propio.

Meses 9-10

Con OpenTelemetry, metricas y load testing empiezas a optimizar basandote en evidencia, no en intuicion.

Parte 6 - Proyectos de escala extrema

Estas ideas requieren dominar primero Spring Boot, bases de datos, testing, redes basicas, Docker y fundamentos de sistemas distribuidos.

Sistema bancario distribuido

Necesita particionamiento, consistencia, ledger, reconciliacion, observabilidad y una estrategia explicita para fallos entre regiones.

Exchange

Requiere estructuras de datos de alta eficiencia, concurrencia y un modelo determinista para matching.

Cloud platform

Implica control plane, workers, estados asincronos, quotas y automatizacion de infraestructura.

IA platform

Requiere orquestacion, RAG, limites de recursos, sandboxing, tool calling y trazas de ejecucion.

Delivery con tracking global

Combina geolocalizacion, streams de eventos, WebSockets y dispatch.

Streaming platform

Necesita catalogo, event ingestion, cache, search y paths de lectura altamente escalables.

Prerequisito: construir uno de estos proyectos de forma honesta no significa implementar toda una empresa en solitario. Significa crear un modelo de laboratorio que reproduzca algunos problemas relevantes y documentar las decisiones.

Parte 7 - Combinar Spring Boot + Flutter

Flutter puede ser el cliente movil/web y Spring Boot la API de negocio. Esta combinacion es especialmente potente para proyectos personales porque permite demostrar tanto producto como backend.

Flutter

- > REST API
- > Spring Boot
- > PostgreSQL

Flutter

- > API Gateway
- > Spring Boot services
- > Kafka
- > PostgreSQL / Redis

Buenos candidatos

Banco Digital, Wallet, Delivery, Chat, Plataforma Educativa, Personal Finance, Marketplace y LMS.

Que mostrar

JWT, refresh/expiry, estados de carga, errores tipados, paginacion, uploads, WebSockets, notificaciones y manejo offline cuando tenga sentido.

Parte 8 - Ideas para agregar IA

La IA aporta mas valor cuando resuelve una tarea del dominio que cuando solo agrega un chat generico.

Uso de IA	Ejemplos de proyectos	Donde puede vivir
Fraud detection	Banco, Wallet, Payments	Servicio de riesgo
Recommendation engine	Streaming, musica, educacion	Servicio de recomendacion
Document analysis	RAG, LMS, Legal-like portals	Servicio de ingestion/IA
Code review	AI Code Review	Worker de analisis
Semantic search	Marketplace, cursos, comunidades	Search/embedding service
Forecasting	Finanzas personales, inventario	Analytics service
Classification	Categorias, tickets, moderacion	Async worker

```
Spring Boot
-> AI Service
-> Python / Model
```

Separar el servicio de IA tiene sentido cuando requiere un runtime distinto, librerias de ML, despliegue independiente o escalado diferente. No hay necesidad de introducir una frontera de red solo por usar un modelo.

Parte 9 - Tabla final de comparacion

La tabla compara la naturaleza tecnica de cada proyecto. No declara cual es mejor ni asigna un ranking.

#	Proyecto	Categoria	Backend	DB	Security	Realtime	IA	Eventos	Microservices	Dificultad
1	Banco Digital	FinTech	Spring Boot	PostgreSQL	Alta	Opcional	Opcional	Util	Selectivo	Avanzado
2	Billetera Digital	FinTech	Spring Boot	PostgreSQL	Alta	Opcional	Opcional	Si	Selectivo	Avanzado
3	Plataforma de Pagos	FinTech	Spring Boot	PostgreSQL	Alta	Opcional	Opcional	Util	Selectivo	Avanzado
4	Sistema de Prestamos	FinTech	Spring Boot	PostgreSQL	Alta	Opcional	Opcional	Si	Selectivo	Avanzado
5	Personal Finance Manager	FinTech	Spring Boot	PostgreSQL	Alta	Opcional	Opcional	Util	Selectivo	Intermedio/ Avanzado
6	Plataforma de Inversiones	FinTech	Spring Boot	PostgreSQL	Alta	Opcional	Opcional	Si	Selectivo	Avanzado
7	Exchange / Trading Platform Simulada	FinTech	Spring Boot	PostgreSQL	Alta	Si	Opcional	Util	Posible	Muy Avanzado
8	Sistema Antifraude	FinTech	Spring Boot	PostgreSQL	Alta	Opcional	Si	Si	Selectivo	Avanzado
9	Marketplace tipo MercadoLibre	E-commerce	Spring Boot	PostgreSQL	Alta	Opcional	Opcional	Util	Selectivo	Avanzado
10	Tienda Online Modular	E-commerce	Spring Boot	PostgreSQL	Alta	Opcional	Opcional	Si	Selectivo	Intermedio/ Avanzado
11	Plataforma de Suscripciones	E-commerce	Spring Boot	PostgreSQL	Alta	Opcional	Opcional	Util	Selectivo	Avanzado
12	Sistema de Delivery	E-commerce	Spring Boot	PostgreSQL	Alta	Si	Opcional	Si	Selectivo	Avanzado
13	Gestion de Inventario	E-commerce	Spring Boot	PostgreSQL	Alta	Opcional	Opcional	Util	Selectivo	Avanzado
14	Plataforma B2B de Compras	E-commerce	Spring Boot	PostgreSQL	Alta	Opcional	Opcional	Si	Selectivo	Avanzado
15	Sistema de Reservas y Cupones	E-commerce	Spring Boot	PostgreSQL	Alta	Opcional	Opcional	Si	Selectivo	Avanzado
16	Red Social	Plataformas	Spring Boot	PostgreSQL	Alta	Opcional	Opcional	Si	Selectivo	Avanzado
17	Plataforma de Chat	Plataformas	Spring Boot	PostgreSQL	Alta	Si	Opcional	Util	Selectivo	Avanzado
18	Discord Simplificado	Plataformas	Spring Boot	PostgreSQL	Alta	Opcional	Opcional	Si	Selectivo	Avanzado
19	Plataforma de Comunidades	Plataformas	Spring Boot	PostgreSQL	Alta	Opcional	Opcional	Util	Selectivo	Avanzado
20	Sistema de Notificaciones en Tiempo Real	Plataformas	Spring Boot	PostgreSQL	Alta	Opcional	Opcional	Si	Selectivo	Avanzado
21	Plataforma Educativa	Educacion	Spring Boot	PostgreSQL	Alta	Opcional	Opcional	Util	Selectivo	Intermedio/ Avanzado
22	Sistema de Cursos Online	Educacion	Spring Boot	PostgreSQL	Alta	Opcional	Opcional	Si	Selectivo	Avanzado

23	LMS Empresarial	Educacion	Spring Boot	PostgreSQL	Alta	Opcional	Opcional	Util	Selectivo	Avanzado
24	Plataforma de Examenes	Educacion	Spring Boot	PostgreSQL	Alta	Opcional	Opcional	Si	Selectivo	Avanzado
25	Sistema de Gestion Universitaria	Educacion	Spring Boot	PostgreSQL	Alta	Opcional	Opcional	Si	Selectivo	Avanzado
26	Sistema de Seguimiento Academico	Educacion	Spring Boot	PostgreSQL	Alta	Opcional	Opcional	Si	Selectivo	Avanzado
27	AI Coding Platform	IA	Spring Boot	PostgreSQL	Alta	Opcional	Si	Util	Posible	Muy Avanzado
28	Asistente de Programacion	IA	Spring Boot	PostgreSQL	Alta	Opcional	Si	Si	Selectivo	Avanzado
29	Plataforma de Ejecucion deCodigo	IA	Spring Boot	PostgreSQL	Alta	Opcional	Si	Util	Posible	Muy Avanzado
30	RAG sobre Documentos	IA	Spring Boot	PostgreSQL	Alta	Opcional	Si	Si	Posible	Muy Avanzado
31	Plataforma de Agentes IA	IA	Spring Boot	PostgreSQL	Alta	Opcional	Si	Util	Posible	Muy Avanzado
32	AI Code Review Platform	IA	Spring Boot	PostgreSQL	Alta	Opcional	Si	Si	Selectivo	Avanzado
33	Uber Simplificado	Sistemas	Spring Boot	PostgreSQL	Alta	Si	Opcional	Util	Posible	Muy Avanzado
34	Airbnb Simplificado	Sistemas	Spring Boot	PostgreSQL	Alta	Opcional	Opcional	Si	Selectivo	Avanzado
35	Sistema tipo Netflix	Sistemas	Spring Boot	PostgreSQL	Alta	Opcional	Si	Util	Selectivo	Avanzado
36	Sistema tipo Spotify	Sistemas	Spring Boot	PostgreSQL	Alta	Opcional	Si	Si	Selectivo	Avanzado
37	Plataforma tipo Steam	Sistemas	Spring Boot	PostgreSQL	Alta	Opcional	Opcional	Util	Selectivo	Avanzado
38	Mini AWS	Sistemas	Spring Boot	PostgreSQL	Alta	Opcional	Opcional	Si	Posible	Muy Avanzado
39	Sistema de Observabilidad	Sistemas	Spring Boot	PostgreSQL	Alta	Si	Opcional	Si	Posible	Muy Avanzado
40	Plataforma de Deployment	Sistemas	Spring Boot	PostgreSQL	Alta	Opcional	Opcional	Si	Posible	Muy Avanzado
41	Sistema Global de Pagos	Sistemas	Spring Boot	PostgreSQL	Alta	Opcional	Opcional	Util	Posible	Muy Avanzado
42	Sistema de Event Streaming	Sistemas	Spring Boot	PostgreSQL	Alta	Opcional	Opcional	Si	Selectivo	Avanzado

Parte 10 - 24 ideas adicionales de 1 a 4 semanas

URL Shortener

Servicio pequeño para acortar URLs, resolver redirects y medir clicks. Practica cache, claves unicas, rate limiting y analytics.

Sistema de Tickets

Mesa de ayuda con estados, prioridades, SLA, asignacion y auditoria. Excelente para workflows y permisos.

Password Manager

Vault personal educativo con cifrado del lado cliente, metadata protegida y control de acceso. El reto esta en seguridad y threat modeling.

File Storage API

API para subir, descargar y compartir archivos con metadatos, expiracion y permisos. Practica object storage y signed URLs.

Notification Service

Servicio reutilizable de email/in-app/push con preferencias y retries. Ideal para Kafka y DLQ.

Geolocation API

API que normaliza coordenadas, busca lugares y ofrece calculos geograficos. Practica indices y cache.

Statistics API

Servicio que recibe eventos y produce agregados. Practica ventanas temporales y procesamiento batch.

Logging Service

Pequena plataforma que centraliza logs de otras apps y permite filtrarlos. Buen ejercicio de ingestion.

Authentication Service

Servicio dedicado para registro, login, refresh tokens, roles y recovery. Ideal para profundizar Spring Security.

Feature Flag Service

Gestion de flags por entorno, usuario o porcentaje. Practica cache, consistencia y reglas de evaluacion.

Config Service

Backend para configuraciones versionadas de aplicaciones. Puede crecer con rollout y audit log.

Job Scheduler

API para crear jobs recurrentes y workers que los ejecutan. Practica locks, retries y estados.

Webhook Gateway

Servicio que recibe, valida, almacena y reintenta webhooks de terceros. Excelente para idempotencia.

Audit Log Platform

Sistema central para registrar cambios administrativos. Practica append-only logs, retencion y query.

Search API

Backend de busqueda con filtros y paginacion. Puede evolucionar desde PostgreSQL full text hasta OpenSearch.

Inventory Importer

Pipeline que importa CSV/Excel, valida filas, reporta errores y actualiza stock. Practica batch y procesamiento incremental.

Expense Splitter

Aplicacion para dividir gastos entre amigos con balances y settlements. Practica reglas de dinero y grafos sencillos.

Study Planner

Organizador de materias, sesiones de estudio y recordatorios. Buen puente entre CRUD y scheduling.

Interview Practice API

Banco de preguntas, sesiones, scoring y feedback. Puede integrarse con IA para generar ejercicios.

Pastebin con Expiracion

Almacena texto con TTL y links publicos/privados. Practica cache y expiracion.

Polling Service

Sistema para crear encuestas, limitar votos y generar resultados. Practica concurrencia y agregados.

Mini CMS

Backend para contenido versionado con drafts, publishing y roles. Practica state transitions y audit.

Link Preview Service

Genera metadata de URLs de forma asincrona. Practica crawling basico, timeouts, cache y SSRF defenses.

API Usage Metering

Cuenta requests por API key, aplica cuotas y genera reportes. Buen proyecto para Redis, counters y rate limits.

Checklist final para elegir un proyecto

Pregunta 1: Que problema resuelve y para quien?

Pregunta 2: Cual es el MVP mas pequeno que todavia demuestra la idea?

Pregunta 3: Que reglas de negocio lo diferencian de un CRUD?

Pregunta 4: Que datos deben ser consistentes y cuales pueden ser eventualmente consistentes?

Pregunta 5: Donde aparecen concurrencia, retries, cache, eventos o procesos largos?

Pregunta 6: Que modulo podria convertirse mas adelante en servicio independiente y por que?

Checklist de portfolio

```
README
Modelo ER / diagrama de dominio
OpenAPI / ejemplos de requests
Tests unitarios + integracion
Docker Compose
CI/CD
Observabilidad minima
Screenshots / demo
Decisiones de arquitectura
Casos de fallo reproducibles
```

Un proyecto personal convincente no necesita fingir ser una plataforma global. Necesita demostrar que entiendes el problema, que puedes construir una solucion completa y que sabes explicar por que tomaste determinadas decisiones.